

32555 – Fundamentals of Software Development



Welcome to 32555 – Fundamentals of Software Development

Your Teaching Team:

Dr. Yining Hu | Subject Coordinator

Yining is a Lecturer in Software Engineering. She works with many SMEs to develop software and AI solutions through student and research projects at UTS.

Contact: yining.hu@uts.edu.au

Dr. Xiaojie (Zoe) Lin | Tutor

Zoe is a full-stack software engineer with an interest in data analysis, specializing in JAVA, R, Python, JavaScript, PHP, Bash, and MATLAB; familiar with AWS.

Contact: Xiaojie.Lin@uts.edu.au

Dr. Farshid Keivanian | Tutor

Dr. Farshid Keivanian is an experienced university lecturer and researcher with a PhD in Information Technology. He has over 15 years of teaching experience across Australian institutions, specialising in software engineering, data science, AI, and cybersecurity.

Contact: Farshid.Keivanian@uts.edu.au

Yong Pov | Tutor

Yong is a full-stack software engineer with experience in React, JavaScript, TypeScript, Java, Python, SQL, and AI-assisted learning platforms.

Contact: Yong.Pov@uts.edu.au

Welcome to 32555 – Fundamentals of Software Development

Cmp1 #	Location	Time	Tutor
01	CB11.B1.400	Fri 16:00-18:00	Xiaojie (Zoe) Lin
07	CB11.B1.401	Wed 10:00-12:00	Farshid Keivanian
08	CB11.B1.400	Fri 12:00-14:00	Yong Pov
09	CB11.B1.103	Wed 16:00-18:00	Farshid Keivanian
10	CB11.B1.401	Wed 12:00-14:00	Farshid Keivanian

Welcome to 32555 Fundamentals of Software Development.

During this subject you will learn how software is:

- Analysed
- Designed
- Developed
- Tested

using

- Java
- Python
- UML (UML = Unified Modeling Language)

By the end of the semester you will be able to build a small software system from beginning to end.

Welcome to 32555 Fundamentals of Software Development.

UML is a standard visual language used to plan and design software before writing code.

Example

Instead of writing code immediately,
we first draw diagrams showing

- users
- classes
- relationships
- system behaviour

This helps developers understand the software before implementation.

What will we learn?

This subject covers the complete software development process.

Topics include:

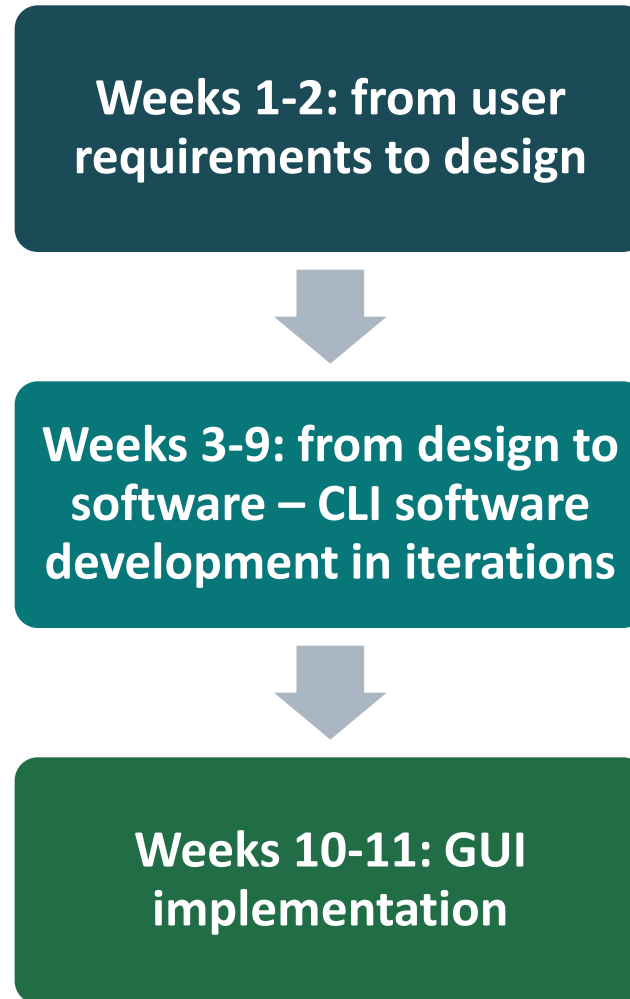
- Software Development Process
- Requirements Analysis
- UML Modelling
- Object-Oriented Programming
- Java Programming
- Python Programming
- GUI Development
- File Handling
- Software Testing

Subject Learning Outcomes

By the end of this subject you should be able to:

- ✓ Analyse software requirements
- ✓ Design software using UML
- ✓ Develop software using Java or Python
- ✓ Test software
- ✓ Build software that satisfies user requirements

Subject Structure



Weekly Learning Style

Google >> Farshid Keivanian GitHub

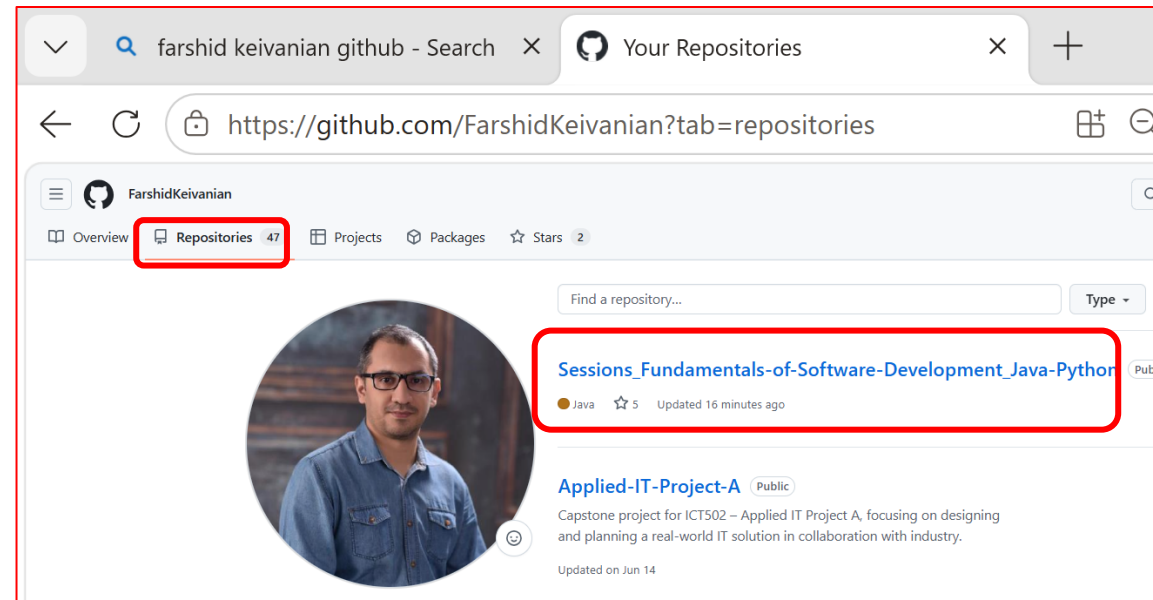
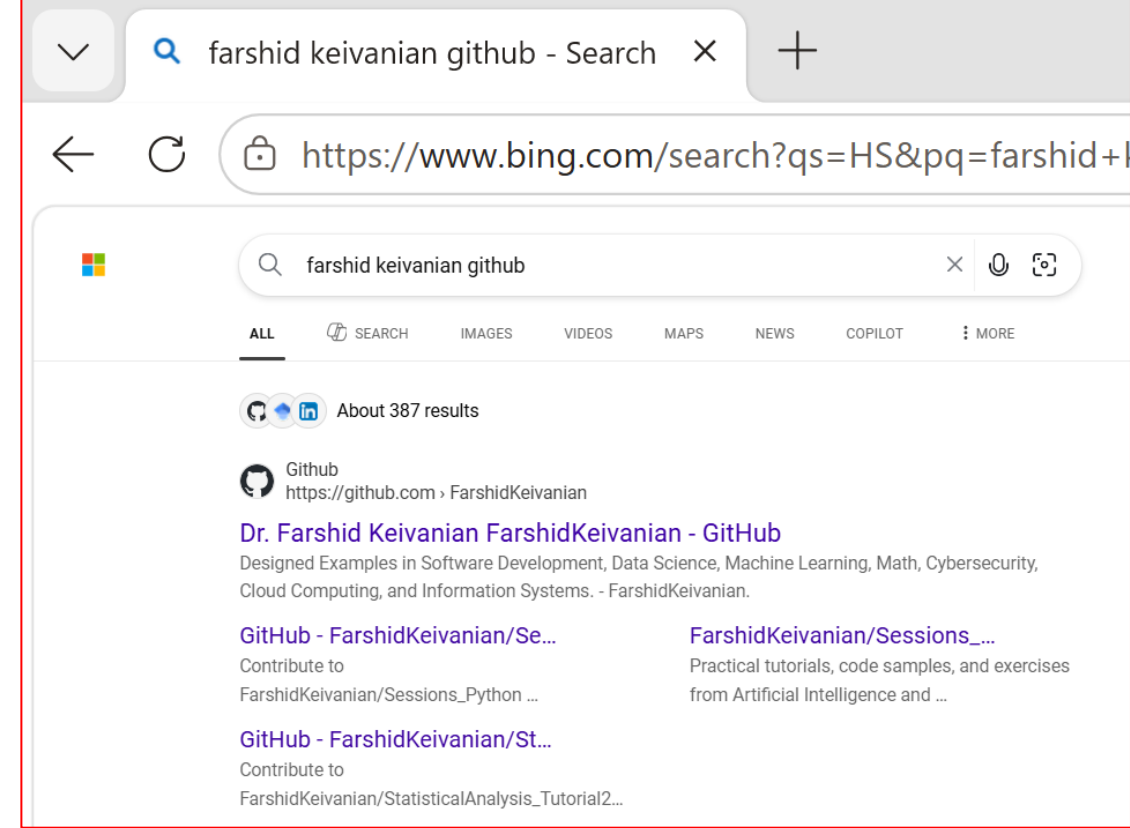
- Repository >> Sessions_Fundamentals-of-Software-Development_Java-Python

It includes:

- Weekly examples
- Java examples
- Python examples
- OOP examples
- Practice programs
- Additional code discussed in class

I will continue uploading new materials every week.

Please check the repository regularly.



Weekly Learning Style

Before class

- Read slides

During workshop

- Mini lecture
- Live coding
- UML examples
- Practice activities
- Questions and discussion

After class

- Practice programming
- Review slides
- Complete exercises

Assessment Overview

Assessment	Weight
Assessment 1 – Case Study	70%
Quiz 1	15%
Quiz 2	15%

Assessment Overview

Quiz 1 (15)

- Week 7; in-class; content of Weeks 1-2

Quiz 2 (15)

- Week 12; in-class; content of Weeks 3-11

Project (70):

- Part 1 (end of Week 7): requirement analysis (group, Sparkplus peer rating)
- Part 2 (end of Week 13): software development (individual)
- Part 3 (Week 12 & Week 13): Showcase

Assessment 1 (70%)

This is the major assessment.

It is completed in groups. **Select your group in weeks 1 to 4**

It contains three parts.

Part 1 (15%)

Analysis and Design

Week 7

Includes

- Requirements
- Use Cases
- UML Use Case Diagram
- UML Class Diagram

The screenshot shows the Canvas LMS interface for the course '32555 Fundamentals of Software Development - Spring 2026'. The page is titled 'Groups' and shows a list of groups under the 'Everyone' tab. The groups are labeled 'Cmp1-01' through 'Cmp1-10'. The 'Cmp1-01' group is expanded, showing a list of 10 groups, each with 0/4 students. The 'Unassigned Students (174)' list is also visible, showing a search bar and a list of student names with plus icons to add them to a group.

Unassigned Students (174)	Groups (10)
Search users	Cmp1-01 1 0 / 4 students
Ariharasudhan Adaikalam +	Cmp1-01 2 0 / 4 students
Sneha Adhikary +	Cmp1-01 3 0 / 4 students
Mumin Ahmed +	Cmp1-01 4 0 / 4 students
Hongyi Ai +	Cmp1-01 5 0 / 4 students
Hani Allam +	Cmp1-01 6 0 / 4 students
Nazire Ata +	
Chiranth Ballyameederira +	
Chimedshirchin Batjargal +	
Mathis Arthur Beissner +	
Janhavy Bhalerao +	

Assessment 1 (70%)

Part 2 (50%)

Software Development

Week 13

Develop the software using

- Java OR Python

Implement

- Classes
- File handling
- Exception handling
- CLI (Command-Line Interface)
- GUI (Graphical User Interface)

The screenshot displays the Canvas LMS interface for a course titled "32555 Fundamentals of Software Development - Spring 2026". The page is titled "Groups" and shows a list of groups under the "Everyone" tab. The groups are labeled "Cmp1-01" through "Cmp1-10". Each group has a status of "0 / 4 students". The interface includes a sidebar with navigation links such as "Home", "Subject Information", "Reading List", "Announcements", "Modules", "Assignments", "Quizzes", "Grades", "Media Gallery", "People", "Search", "GradesPlus", "Discussions", and "Rubrics". The "People" link is currently selected. The main content area shows a search bar for "Unassigned Students (174)" and a list of students with their names and a plus sign to add them to a group. The groups list on the right shows the group name, the number of students (0 / 4), and a plus sign to add more students.

Unassigned Students (174)	Groups (10)
Search users	
☐ Ariharasudhan Adaikalam	▶ Cmp1-01 1 0 / 4 students
☐ Sneha Adhikary	▶ Cmp1-01 2 0 / 4 students
☐ Mumin Ahmed	▶ Cmp1-01 3 0 / 4 students
☐ Hongyi Ai	▶ Cmp1-01 4 0 / 4 students
☐ Hani Allam	▶ Cmp1-01 5 0 / 4 students
☐ Nazire Ata	▶ Cmp1-01 6 0 / 4 students
☐ Chiranth Ballyameederira	
☐ Chimedshirchin Batjargal	
☐ Mathis Arthur Beissner	
☐ Janhavy Bhalerao	

Assessment 1 (70%)

CLI = Command-Line Interface

Simple explanation:

Users interact with software by typing commands.

```
Example  
  
1. Add Student  
2. Search Student  
3. Exit  
  
Enter your choice:
```

There are no buttons.

Everything is entered using the keyboard.

Assessment 1 (70%)

GUI = Graphical User Interface

Example

Windows

Buttons

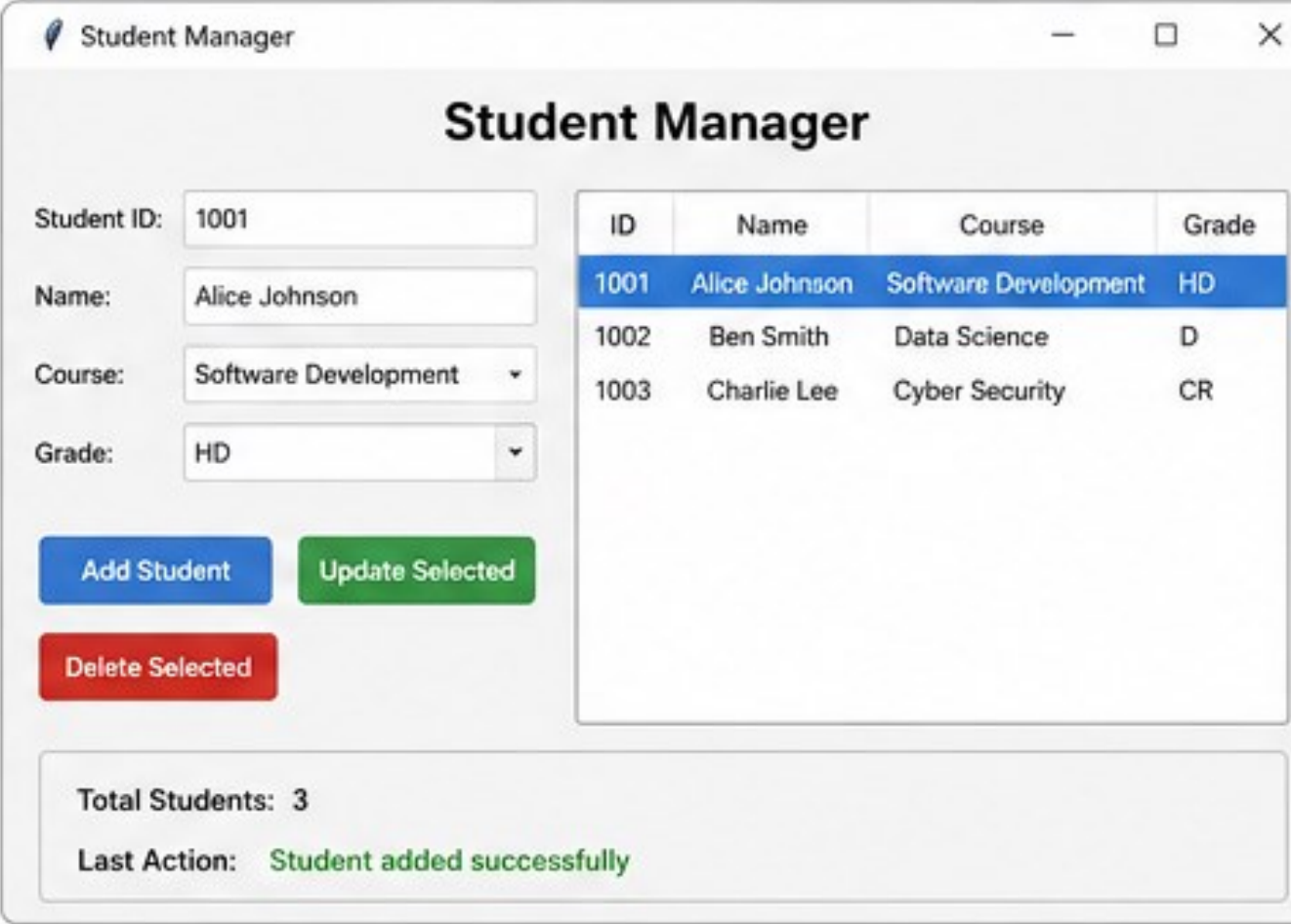
Menus

Icons

Mouse clicks

Most desktop applications use a GUI.

EXAMPLE GUI (PYTHON TKINTER)



The screenshot shows a window titled "Student Manager" with a standard macOS-style title bar (red, yellow, and green buttons). The window content is titled "Student Manager" in bold. On the left, there are four input fields: "Student ID:" with the value "1001", "Name:" with "Alice Johnson", "Course:" with a dropdown menu showing "Software Development", and "Grade:" with a dropdown menu showing "HD". Below these fields are three buttons: "Add Student" (blue), "Update Selected" (green), and "Delete Selected" (red). On the right, there is a table with four columns: "ID", "Name", "Course", and "Grade". The table contains three rows of data. The first row is highlighted in blue. At the bottom of the window, there is a status bar with two labels: "Total Students: 3" and "Last Action: Student added successfully".

ID	Name	Course	Grade
1001	Alice Johnson	Software Development	HD
1002	Ben Smith	Data Science	D
1003	Charlie Lee	Cyber Security	CR

Total Students: 3
Last Action: Student added successfully

Library used: Tkinter (comes with Python standard library)
No extra installation required.

Assessment 1 (70%)

Part 3 (5%)

Project Showcase

Week 13 (19 to 23 Oct)

Each student
demonstrates their
software during lab.

← → ↺ 🔍 🔖 ☆ 🔔 uts.edu.au/for-students/current-students/managing-you...		
Important dates	Results publication date	Monday 29 June 2026
Academic year dates	Rescheduled and alternative exam period	Monday 6 – Friday 10 July 2026
2026 Academic year dates		
2027 Academic year dates		
Census date	Main Calendar: Spring Session	
Centrally conducted exams dates and timetables	Orientation	Monday 20 – Friday 24 July 2026
Graduation	Session commences	Monday 27 July 2026
Last day to enrol	Last day to enrol	Monday 10 August 2026
Other important dates	Census date	Thursday 27 August 2026
Principal dates	Mid-session StuVac (study vacation)	Monday 21 – Friday 25 September 2026
Tuition payment dates	Formal classes end	Friday 23 October 2026
	Final StuVac (study vacation)	Monday 26 – Friday 30 October 2026
	Final assessment period	Monday 2 – Friday 20 November 2026
	Results publication date	Monday 7 December 2026
	Rescheduled and alternative exam period	Monday 14 – Friday 18 December 2026

Quiz 1 (15%)

Week 7

Individual

30 minutes

Single attempt

Restricted Open Book

Only subject materials may be used.

Topics

No Generative AI.

- Software Development
- SDLC
- Requirements
- UML
- Classes
- Objects

No collaboration.

Quiz 2 (15%)

Week 12

Individual

30 minutes

Topics

Programming

- Java

or

- Python

(depending on your stream)

Passing the Subject

To pass this subject

You must

- achieve at least 50% overall
- submit assessments before deadlines
- maintain academic integrity

Academic Integrity

Your work must be your own.

You may

- ✓ discuss ideas
- ✓ ask questions
- ✓ attend consultations

You may NOT

- ✗ copy another student's work
- ✗ use Generative AI during restricted quizzes
- ✗ submit someone else's code

Software Development Life Cycle (SDLC)

Software is usually developed step by step.

Example

Imagine we want to build a Library Management System.

1. Requirements

What does the customer need?

Example

- Borrow books
- Return books
- Search books

Software Development Life Cycle (SDLC)

2. Analysis

Understand the problem.

Questions:

Who will use the system?

Students?

Librarians?

Administrators?

Software Development Life Cycle (SDLC)

3. Design

Plan the solution.

Draw

- Use Case Diagram
- Class Diagram

Software Development Life Cycle (SDLC)

4. Coding

Write the software in

- Java
- or
- Python

Software Development Life Cycle (SDLC)

5. Testing

Check whether everything works correctly.

Example

Does "Borrow Book" work?

Can invalid input be detected?

Software Development Life Cycle (SDLC)

6. Deployment

Install the software for users.

Example

Install it in the university library.

7. Maintenance

Fix bugs.

Add new features.

Example

Later the library requests:

"Please add online reservations."

Why do we need a Software Development Methodology?

Imagine building a house.

Would you start building before making a plan?

No.

Software is the same.

A methodology provides a structured way to build software successfully.

Software Development Methodologies

A methodology is how we organise the software development process.

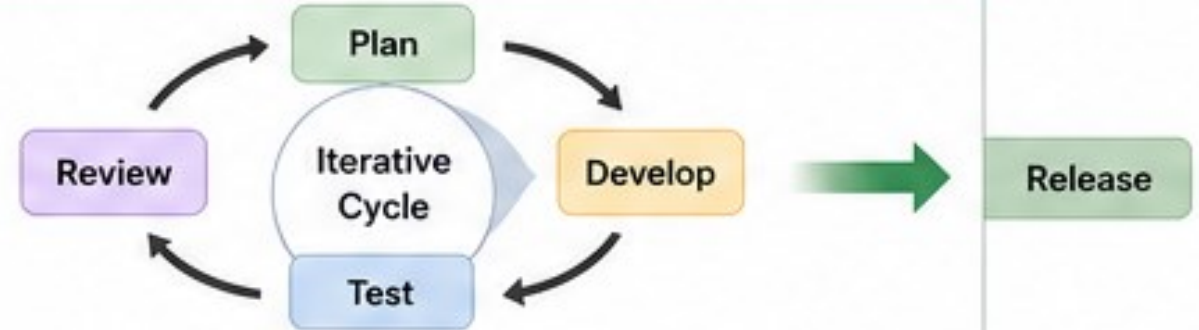
SOFTWARE DEVELOPMENT METHODOLOGIES

WATERFALL (Traditional)



- One stage finishes before the next begins.
- Changes later are difficult and costly.
- Suitable when requirements are stable and clear.

AGILE (Modern)



- Work in small iterations (cycles).
- Get feedback early and often.
- Easy to adapt to changes.
- Frequent software releases.

Object-Oriented Programming

This subject focuses on Object-Oriented Programming (OOP).

Main concepts

- Classes
- Objects
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

We will learn these step-by-step.

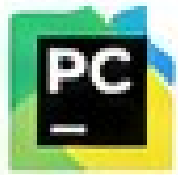
Software Development Tools

During this semester we will use

- Python
- Java
- VS Code / IDE
- UML tools
- Canvas
- GitHub



Visual Studio
Code



PyCharm /
IntelliJ IDEA



Git &
GitHub



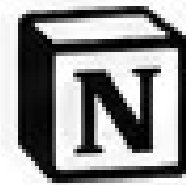
Draw.io /
Lucidchart



Postman



SQLite /
DB Browser



Notion



Microsoft
Excel

Software Development Tools

What is Postman used for?

Postman is a tool for testing APIs (Application Programming Interfaces).

Suppose a program provides weather information.

Instead of opening a website, Postman can send a request directly to the server.

Example:

GET

<https://api.weather.com/current>

The server replies:

```
{  
  "temperature":22,  
  "city":"Sydney"  
}
```



Visual Studio
Code



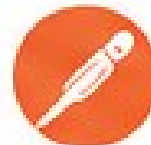
PyCharm /
IntelliJ IDEA



Git &
GitHub



Draw.io /
Lucidchart



Postman



SQLite /
DB Browser



Notion



Microsoft
Excel

Software Development Tools

What is Notion?

Notion is an all-in-one workspace used to organise information, notes, projects, and team collaboration.

Think of it as a combination of:

- Microsoft Word
- OneNote
- Trello
- Planner
- Wiki

all in one application.



Visual Studio
Code



PyCharm /
IntelliJ IDEA



Git &
GitHub



Draw.io /
Lucidchart



Postman



SQLite /
DB Browser



Notion



Microsoft
Excel

Software Development Tools

Students do not need to use for this unit. What can students use Notion for?

- Organising lecture notes
- Managing assessment deadlines
- Planning a group project
- Recording meeting notes
- Creating a personal study planner
- Tracking weekly progress
- Writing project documentation



Visual Studio
Code



PyCharm /
IntelliJ IDEA



Git &
GitHub



Draw.io /
Lucidchart



Postman



SQLite /
DB Browser



Notion



Microsoft
Excel

Software Development Tools

Why do software developers use Postman?

Postman allows developers to:

- Test REST APIs
- Send GET requests
- Send POST requests
- Update data (PUT/PATCH)
- Delete data (DELETE)
- View JSON responses
- Test authentication (API keys, tokens)
- Debug backend services



Visual Studio
Code



PyCharm /
IntelliJ IDEA



Git &
GitHub



Draw.io /
Lucidchart



Postman



SQLite /
DB Browser



Notion



Microsoft
Excel

Software Development Tools

Will students use Postman in this unit?

Based on the subject outline, no.

This subject focuses on:

Software development

- UML
- Java
- Python
- CLI
- GUI
- Testing

No need to build or test web APIs.



Visual Studio
Code



PyCharm /
IntelliJ IDEA



Git &
GitHub



Draw.io /
Lucidchart



Postman



SQLite /
DB Browser



Notion



Microsoft
Excel

Software Development Tools

For this unit, I would recommend:

Required

- Canvas
- Python 3
- Java JDK
- VS Code or IntelliJ IDEA
- GitHub (recommended for group work)
- diagrams.net (Draw.io) for UML



Visual Studio
Code



PyCharm /
IntelliJ IDEA



Git &
GitHub



Draw.io /
Lucidchart



Postman



SQLite /
DB Browser



Notion



Microsoft
Excel

Software Development Tools

Useful

- GitHub Desktop
- Notepad++
- Microsoft Office

Later (for further development)

- JavaFX
- Tkinter
- JUnit
- Maven

This list is more closely aligned with the learning outcomes and assessment tasks for the subject.

Software Development Tools

GitHub Desktop (Recommended for Beginners)

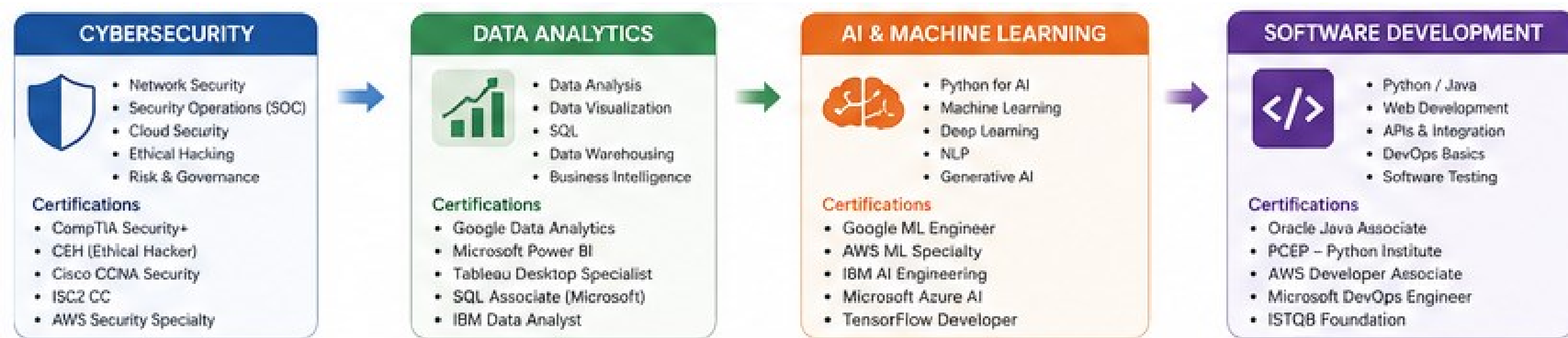
Many first-year and postgraduate students prefer GitHub Desktop because they don't need to remember Git commands.

The typical workflow is:

1. Clone the repository (first time only).
2. Open the project and make changes.
3. GitHub Desktop automatically shows the changed files.
4. Enter a short commit message, such as *"Added login feature"*.
5. Click Commit to main.
6. Click Push origin to upload the changes to GitHub.

Students can complete most common Git tasks using the graphical interface without using the command line.

Roadmap – Highly In Demand Skills (2026 – 2028)



Can I use AI Tools?

Activity	GitHub Copilot	ChatGPT / Other AI
Learning / Practice		
Assignments (Coding / Reports)	 Use with caution and acknowledge	 Use with caution and acknowledge
Presentations	 Use with caution and acknowledge	 Use with caution and acknowledge
Quizzes / Exams	 NOT allowed	 NOT allowed

All assessed work must be your own. Follow UTS academic integrity policy.

Should we use GitHub for the group project?

Yes, I strongly recommend it.

GitHub helps your team:

- Store your project safely online.
- Work together without overwriting each other's code.
- Keep a history of changes.
- Restore previous versions if something goes wrong.
- Demonstrate your contribution to the project.

Many software companies use GitHub every day.

Note: GitHub is recommended for collaboration. Always follow the assessment instructions provided by the Subject Coordinator.

Should we use GitHub for the group project?

Each student:

- writes code
- commits changes
- pushes to GitHub

Everyone can then pull the latest version.

Git vs GitHub

Students often confuse these.

Git is the version control tool. GitHub is the online service that stores Git projects and allows teams to collaborate.

Git	GitHub
Software installed on your computer	Website for hosting Git repositories
Tracks code changes	Stores projects online
Used with the command line or IDE	Used for collaboration
Works offline	Requires internet for syncing

GitHub workflow for a Group Project

The workflow students should remember

1



Create Repository

One team member creates the GitHub repository.



2



Clone Repository

Each team member clones the repository to their computer.



3



Write Code

Work on your assigned tasks and write your code.



4



Commit Changes

Save your changes with a meaningful commit message.



5



Push to GitHub

Upload your committed changes to the GitHub repository.



6



Other teammates Pull the latest version

Your teammates pull the latest changes to their computers.



7



Continue coding

Repeat the cycle and keep your project up to date.

My recommendation

- ✓ Use **GitHub** for the shared group repository.
- ✓ Use **GitHub Desktop** for synchronising work (recommended for beginners).
- ✓ Use **VS Code** or **IntelliJ IDEA** for writing and testing code.

Focus on building your software, collaborating effectively, and keeping your project safe with version control. You will learn more advanced Git features when you need them.

Python Installation

Please install the latest version of Python.

Also install

- VS Code (or another IDE)
- Tkinter support

Installation guides are available on Canvas.

If you experience any problems, let me know during the workshop.

Communication

If you have questions,
please contact me through

- Discussion Forums
- Canvas Inbox
- UTS Email (Farshid.Keivanian@uts.edu.au)

I regularly monitor my emails and will reply as soon as possible.

Week 7 Reminder (include around Week 7)

Quiz 1

Assessment Task 2

Worth 15%

During your scheduled lab

30 minutes

Single attempt

Restricted Open Book

Remember:

- Use the lab computer only.
- You may refer to subject materials only.
- Do not use Generative AI.
- Do not collaborate with other students.
- For code questions, read and analyse the code without running it.

Week 1 Practice Quiz

Question 1

Week 1 Practice Quiz: Question 1

Which of the following best describes a software development methodology?

- A. Software methodology is a way to test functions.
- B. Software methodology is the process used to create programs.
- C. Software methodology is only used in procedural programming.
- D. Software methodology is only used for object-oriented programming.

Week 1 Practice Quiz

Correct answer: B

Software methodology is the process used to create programs.

A software development methodology provides a structured approach for:

- Planning
- Analysing
- Designing
- Coding
- Testing
- Deploying
- Maintaining software

Week 1 Practice Quiz

Examples include Waterfall and Agile.

Why are the other options incorrect?

- A. Incorrect: Testing functions is only one part of software development.
- C. Incorrect: Methodologies are not limited to procedural programming.
- D. Incorrect: Methodologies are not limited to object-oriented programming.

Week 1 Practice Quiz

Question 2

Which of the following is not a characteristic of Agile?

- A. At the end of every Agile cycle, a new version of the software is released.
- B. Agile repeats Software Development Life Cycle steps.
- C. Agile completes the Software Development Life Cycle in sequential order with fixed project requirements.
- D. Agile allows new requirements after a prototype has been released.

Week 1 Practice Quiz

Select the option that does not describe Agile.

Correct answer: C

Agile completes the Software Development Life Cycle in sequential order with fixed project requirements.

This statement describes the Waterfall methodology, not Agile.

Explanation of the options

A. Agile characteristic: Each cycle may produce a new or improved version of the software.

B. Agile characteristic: Planning, development, and testing are repeated during multiple cycles.

Week 1 Practice Quiz

C. Not an Agile characteristic: Agile does not require all stages to be completed once in a fixed sequence.

D. Agile characteristic: New or changed requirements can be considered during later cycles.

Simple example

A customer tests an early version of an online shopping system and asks for a wishlist feature. Agile allows the development team to add this request to a future cycle.

Week 1 Practice Quiz

Question 3

Week 1 Practice Quiz: Question 3

Which of the following is an imperative programming paradigm type?

- A. Mathematical
- B. Logical
- C. Functional
- D. Object-Oriented

Week 1 Practice Quiz

Which of the following is an imperative programming paradigm type ?

- ☐ Mathematical
- ☐ Logical
- ☐ Functional
- ☐ Object-Oriented

Week 1 Practice Quiz

Correct answer: D – Object-Oriented

Imperative programming describes a program as a sequence of instructions that changes the program's state.

Object-oriented programming commonly uses this step-by-step approach through:

- ☐ Objects
- ☐ Methods
- ☐ Variables
- ☐ Assignment statements
- ☐ Control structures

Java and Python both support object-oriented programming.

Week 1 Practice Quiz

Why are the other options incorrect?

- A. Mathematical: This is not normally listed as a main programming paradigm.
- B. Logical: Logic programming is a declarative paradigm. Prolog is an example.
- C. Functional: Functional programming focuses on functions and expressions. Haskell is an example.

Week 1 Practice Quiz

Question 4

Which statement best describes procedural programming?

- A. All system functions can access the same global variables.
- B. Functions are encapsulated with the data.
- C. C++ is a procedural programming language.
- D. Procedural programming organises goals into classes.

Select the best answer from the options provided.

Week 1 Practice Quiz

Intended answer: A

All system functions can access the same global variables.

Traditional procedural programs may use shared global data that can be accessed by different procedures or functions.

Explanation of the options

A. Intended correct answer: Procedural programs may use functions that operate on shared data.

B. Incorrect: Combining data and related operations describes encapsulation in object-oriented programming.

Week 1 Practice Quiz

C. Not the best answer: C++ supports procedural programming, but it is a multi-paradigm language and also strongly supports object-oriented programming.

D. Incorrect: Classes are mainly associated with object-oriented programming.

Important clarification

A clearer definition of procedural programming is:

Procedural programming organises a program into procedures or functions that perform a sequence of instructions.

The use of global variables is possible, but it is not required in every procedural program.

Week 1 Practice Quiz

Question 5

Which statement is not correct?

- A. Object-oriented programs are better designed.
- B. Object-oriented operation definitions are hidden; only behaviour is exposed to other classes.
- C. Object-oriented class data is available to all system functions.
- D. An object-oriented program can create many objects from the same class.

Select the statement that is incorrect.

Week 1 Practice Quiz

Correct answer: C

Object-oriented class data is available to all system functions.

This statement is incorrect because object-oriented programming commonly uses encapsulation to control access to data.

Example

```
private double balance;
```

Because balance is private, other classes cannot access it directly.

Week 1 Practice Quiz

Explanation of the options

- A. Generally correct: Object-oriented design can improve organisation, modularity, maintainability, and reuse.
- B. Correct: Internal implementation details can be hidden while required behaviour is provided through public methods.
- C. Incorrect: Class data should not automatically be available to every function in the system.
- D. Correct: One class can be used to create many objects.

Example

```
Student student1 = new Student();
```

```
Student student2 = new Student();
```

Both objects are created from the same Student class.

Week 1 Practice Quiz

Question 6 –

Which statement best describes software requirements analysis?

- A. Requirements analysis is a way to create software architecture.
- B. Requirements analysis refines and transforms requirements into user stories.
- C. Requirements analysis identifies classes and their attributes.
- D. Requirements analysis is used to create software design.

Select the best answer.

Week 1 Practice Quiz

Correct answer: B

Requirements analysis refines and transforms requirements into user stories.

Requirements analysis helps the development team understand:

- What users need
- What the system must do
- Who will use the system
- What conditions or limitations apply

Week 1 Practice Quiz

Explanation of the options

- A. Incorrect: Software architecture is normally developed during system design.
- B. Correct: User needs may be refined and documented as requirements or user stories.
- C. Incorrect: Identifying classes and attributes is usually part of object-oriented analysis and design.
- D. Incorrect: Requirements analysis provides information for the design, but it is not the same as creating the design.

Example user story

As a library member, I want to search for a book so that I can check whether it is available.

Week 1 Practice Quiz

Question 7 –

Why is documentation testing necessary?

- A. To have morning coffee with stakeholders.
- B. To create the project development schedule.
- C. To identify classes in the software.
- D. To ensure the correctness of the requirements.

Select the best answer.

Week 1 Practice Quiz

Correct answer: D

Documentation testing is necessary to ensure the correctness of the requirements.

Reviewing requirements documentation helps identify:

- Missing information
- Contradictions
- Unclear statements
- Incorrect assumptions
- Requirements that cannot be tested

Week 1 Practice Quiz

Explanation of the options

- A. Incorrect: This is unrelated to documentation testing.
- B. Incorrect: Creating a schedule is a project management activity.
- C. Incorrect: Classes are normally identified during object-oriented analysis and design.
- D. Correct: Reviewing documentation helps ensure that the requirements are clear and correct.

Week 1 Practice Quiz

Example

A requirement states:

The system must support online payments.

The development team should clarify:

- Which payment methods?
- Which currencies?
- What happens if payment fails?
- Is a receipt required?

It is easier to correct these issues before coding begins.

Week 1 Practice Quiz

Question 8 –

Which of the following is not a step in the requirements analysis process?

- A. Create a class diagram.
- B. Create a use-case diagram.
- C. Map user stories to a table.
- D. Convert requirements into user stories.

Select the activity that does not belong directly to requirements analysis.

Week 1 Practice Quiz

Question 8

Which of the following is not a step of the requirement analysis process ?

- ☐ Group of answer choices
- ☐ Create a class diagram
- ☐ Create a use-case diagram
- ☐ Map the user-stories to a table
- ☐ Convert requirements to user-stories

Week 1 Practice Quiz

Correct answer: A

Create a class diagram.

A class diagram is mainly used during object-oriented analysis and system design to represent:

- Classes
- Attributes
- Methods
- Relationships between classes

Week 1 Practice Quiz

Explanation of the options

- A. Correct answer: Creating a class diagram normally belongs to a later analysis or design activity.
- B. Part of requirements analysis: A use-case diagram shows how actors interact with the system.
- C. Part of requirements analysis: A table can help organise and trace user stories.
- D. Part of requirements analysis: User needs can be converted into clear user stories.

Week 1 Practice Quiz

Question 9

Which of the following cannot be an actor in a use-case diagram?

- A. Person
- B. Cash register
- C. Login
- D. Printer

Select the option that is not an actor.

Week 1 Practice Quiz

Correct answer: C – Login

An actor is a person, organisation, external device, or external system that interacts with the software.

Login is an action or system function. It is therefore more suitable as a use case, not an actor.

Explanation of the options

A. Person: Can be an actor, such as a customer or administrator.

B. Cash register: Can be an external device or system interacting with another system.

Week 1 Practice Quiz

C. Login: This is an action performed by an actor.

D. Printer: Can be an external device interacting with the system.

Simple use-case example

Actor: Student

Use case: Log in

System: Student portal

Week 1 Practice Quiz

Question 10

A sequence diagram is an example of a:

- A. Behaviour diagram
- B. Interaction diagram
- C. Structure diagram

Select the most specific answer.

Week 1 Practice Quiz

Correct answer: B – Interaction diagram

A sequence diagram shows:

- Objects or participants
- Messages exchanged between them
- The order in which interactions occur
- How the interaction changes over time

Week 1 Practice Quiz

Explanation of the options

- A. Behaviour diagram: Sequence diagrams are broadly included within UML behaviour diagrams, but this is not the most specific answer.
- B. Interaction diagram: This is the best and most specific classification.
- C. Structure diagram: Structure diagrams show the static parts of a system. A class diagram is an example.

Week 1 Practice Quiz

Simple example

- Customer → ATM: Insert card
- ATM → Customer: Request PIN
- Customer → ATM: Enter PIN
- ATM → Bank System: Validate PIN
- Bank System → ATM: PIN accepted

The sequence diagram shows the order in which these messages are exchanged.

Choosing Your Development Environment (IDE)

IDE	Java	Python	Recommended
VS Code	✓	✓	Recommended (supports both Java and Python)
IntelliJ IDEA Community	✓	Limited (requires plugin)	Excellent for Java
Eclipse IDE	✓	Possible with PyDev plugin	Good for Java
PyCharm Community	✗	✓	Excellent for Python

Choosing Your Development Environment (IDE)

You are welcome to use the IDE that you are most comfortable with, provided it supports the required programming language and you can successfully complete the assessment tasks.

If you are developing in Java, IntelliJ IDEA or Eclipse are both excellent choices.

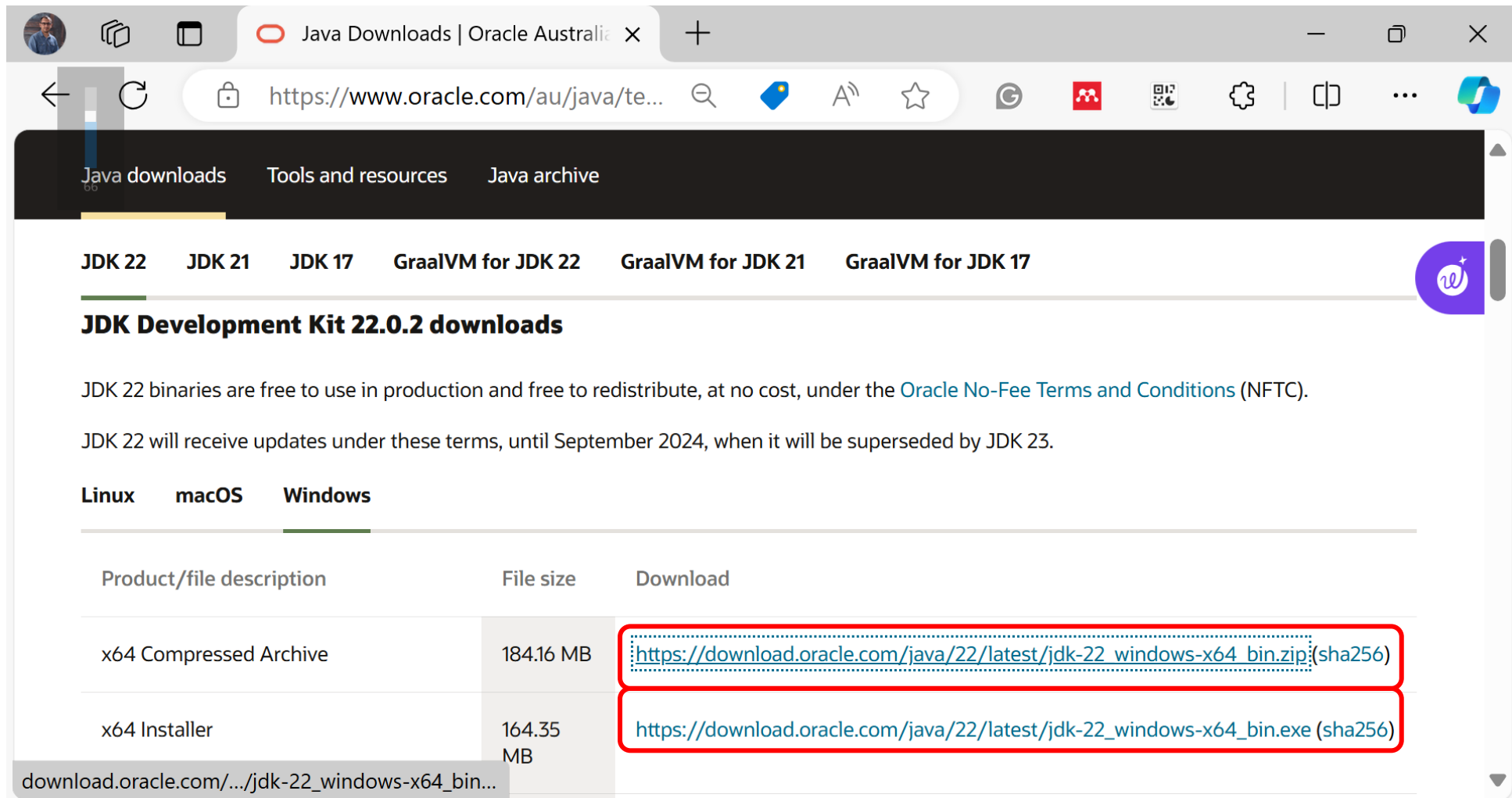
If you are developing in Python, VS Code or PyCharm are recommended.

If you would like to use one IDE for both Java and Python, VS Code is probably the easiest option.

If you choose Eclipse IDE (Java)

- Download and install the idk package

<https://www.oracle.com/au/java/technologies/downloads/#jdk22-windows>



The screenshot shows the Oracle Java Downloads page for JDK 22 on Windows. The page has a dark navigation bar with 'Java downloads', 'Tools and resources', and 'Java archive'. Below this, there are tabs for 'JDK 22', 'JDK 21', 'JDK 17', 'GraalVM for JDK 22', 'GraalVM for JDK 21', and 'GraalVM for JDK 17'. The 'JDK 22' tab is selected, and the page title is 'JDK Development Kit 22.0.2 downloads'. The page content states that JDK 22 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC). It also mentions that JDK 22 will receive updates under these terms, until September 2024, when it will be superseded by JDK 23. Below this, there are tabs for 'Linux', 'macOS', and 'Windows'. The 'Windows' tab is selected, and a table lists the download links for the x64 Compressed Archive and the x64 Installer. The download links are highlighted with red boxes.

Product/file description	File size	Download
x64 Compressed Archive	184.16 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.zip (sha256)
x64 Installer	164.35 MB	https://download.oracle.com/java/22/latest/jdk-22_windows-x64_bin.exe (sha256)

If you choose Eclipse IDE (Java)

- Download and install the idk package

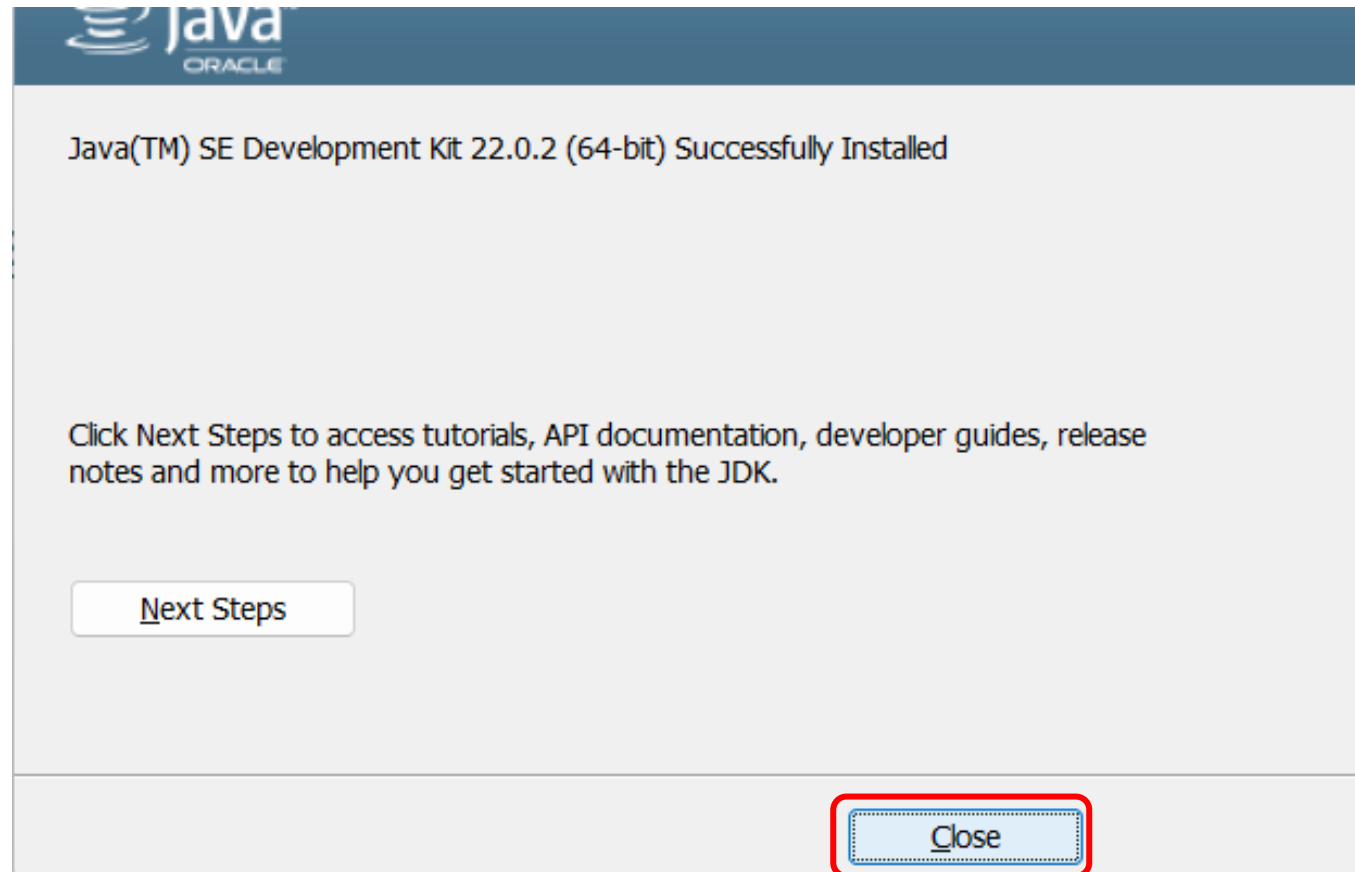
<https://www.oracle.com/au/java/technologies/downloads/#jdk22-windows>



If you choose Eclipse IDE (Java)

- Download and install the idk package

<https://www.oracle.com/au/java/technologies/downloads/#jdk22-windows>



If you choose Eclipse IDE (Java)

- Command Prompt Windows / Terminal in MAC: `java -show-version`

```
C:\Users\gaming 15>java --show-version
java 22.0.2 2024-07-16
Java(TM) SE Runtime Environment (build 22.0.2+9-70)
Java HotSpot(TM) 64-Bit Server VM (build 22.0.2+9-70, mixed mode, sharing)
Usage: java [options] <mainclass> [args...]
        (to execute a class)
or  java [options] -jar <jarfile> [args...]
        (to execute a jar file)
or  java [options] -m <module>[/<mainclass>] [args...]
    java [options] --module <module>[/<mainclass>] [args...]
        (to execute the main class in a module)
or  java [options] <sourcefile> [args]
        (to execute a source-file program)

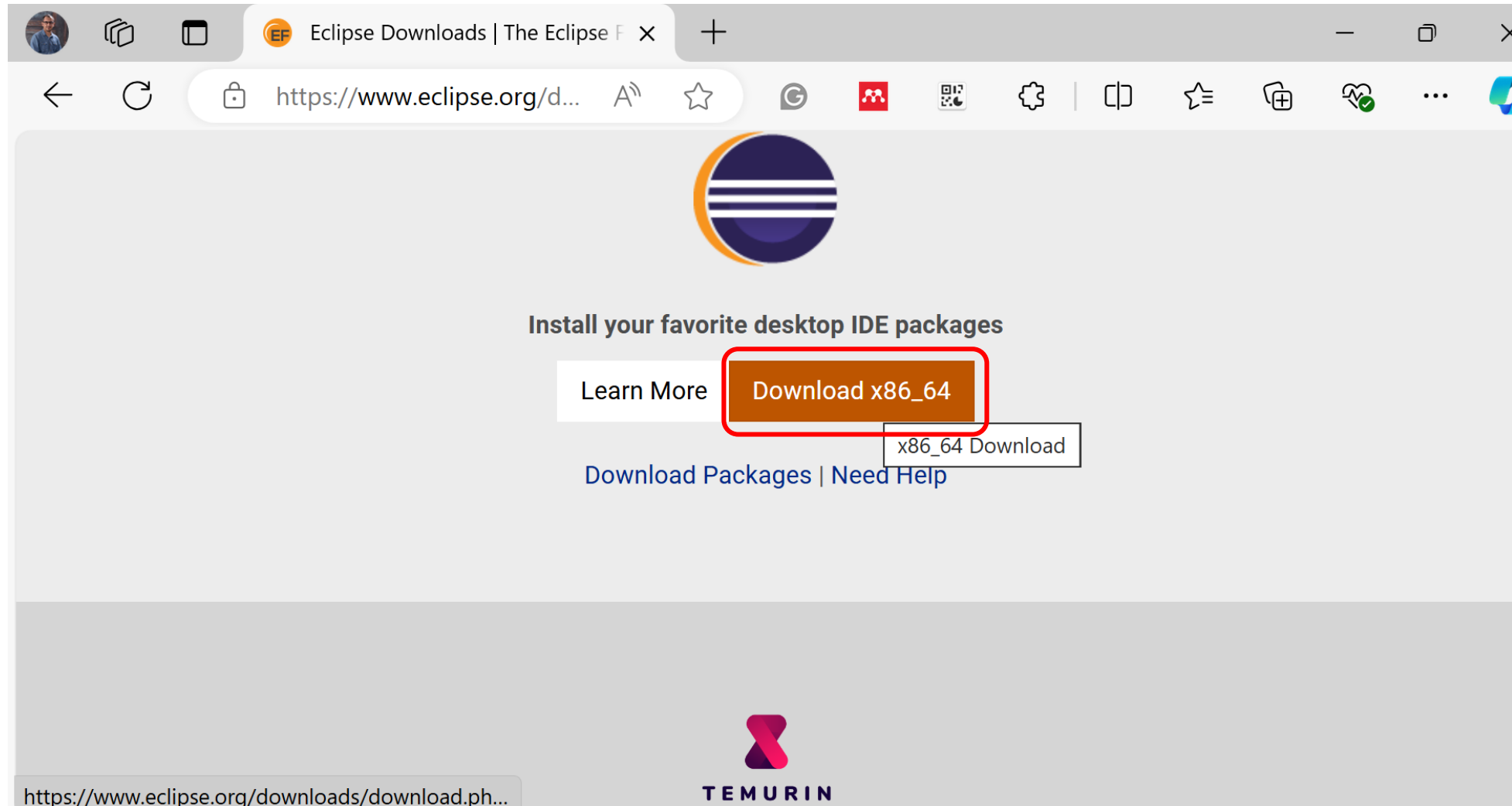
Arguments following the main class, source file, -jar <jarfile>,
-m or --module <module>/<mainclass> are passed as the arguments to
main class.

where options include:

    -cp <class search path of directories and zip/jar files>
```

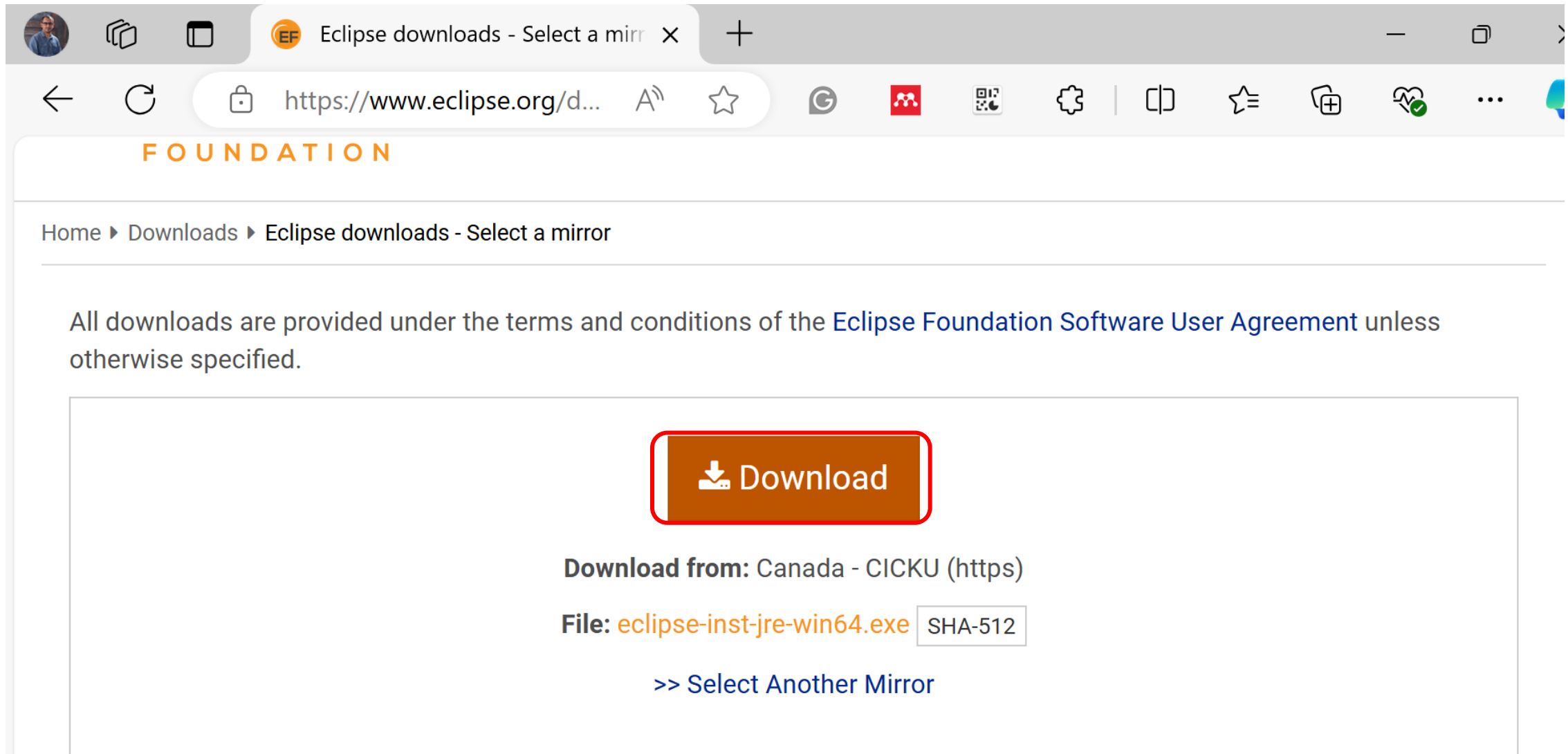
If you choose Eclipse IDE (Java)

- Download Eclipse Software <https://www.eclipse.org/downloads/>



If you choose Eclipse IDE (Java)

- Download Eclipse Software <https://www.eclipse.org/downloads/>



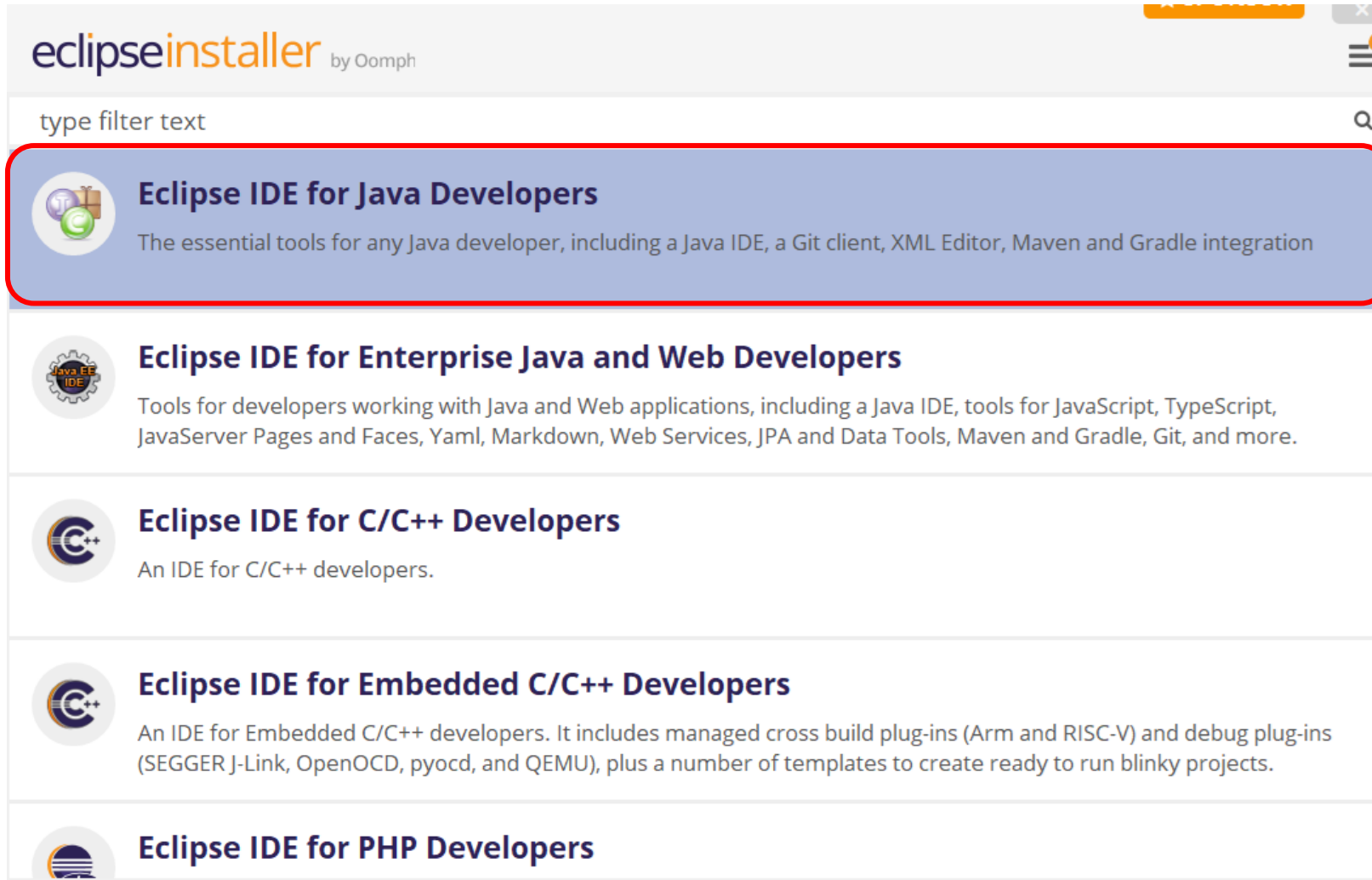
If you choose Eclipse IDE (Java)

- Install Eclipse Software <https://www.eclipse.org/downloads/>

NAME	Date modified	Type	Size
 jdk-22.0.2	04/06/2024 17:25	File folder	
 jdk-22_windows-x64_bin	27/07/2024 07:17	Application	168,292 KB
 eclipse-inst-jre-win64	27/07/2024 07:26	Application	134,526 KB

If you choose Eclipse IDE (Java)

- Install Eclipse Software <https://www.eclipse.org/downloads/>



If you choose Eclipse IDE (Java)

- Install Eclipse Software <https://www.eclipse.org/downloads/>

Artifact download is progressing very slowly from the following hosts: <https://ftp2.osuosl.org>, <https://mirrors.xmission.com>



Eclipse IDE for Java Developers

The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Maven and Gradle integration.

[details](#)

Java 21+ VM

C:\Program Files\Java\jdk-22

Installation Folder

C:\Users\gaming 15\eclipse\java-2024-06

☒ create start menu entry

☒ create desktop shortcut

 INSTALLING



[< BACK](#)

If you choose Eclipse IDE (Java)

- Install Eclipse Software <https://www.eclipse.org/downloads/>

Installation completed successfully. ✕



Eclipse IDE for Java Developers

The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Maven and Gradle integration.

[details](#)

Java 21+ VM

C:\Program Files\Java\jdk-22 ▼ 

Installation Folder

C:\Users\gaming 15\eclipse\java-2024-06 

☒ create start menu entry

☒ create desktop shortcut

▶ LAUNCH

show readme file

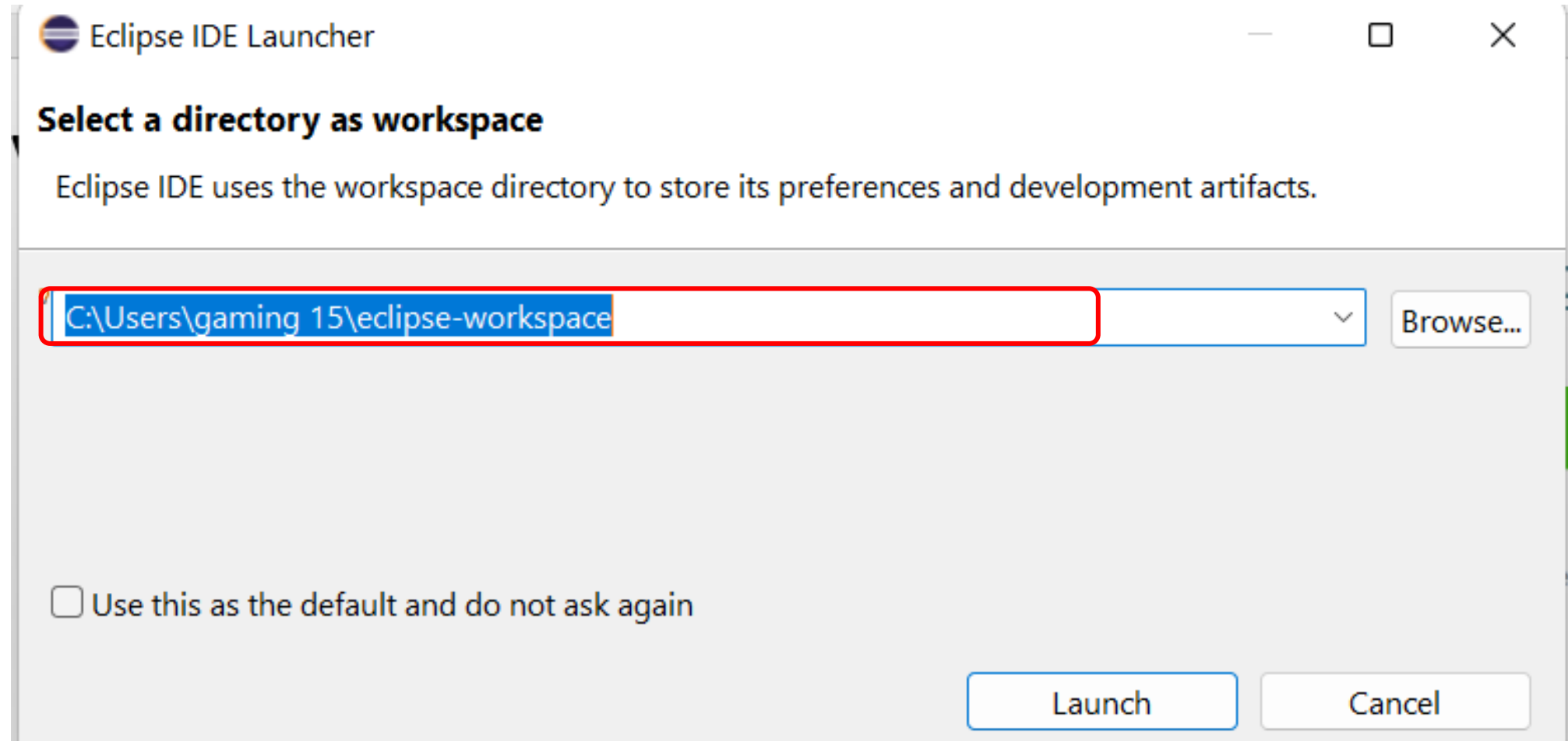
open in system explorer

keep installer

 BACK

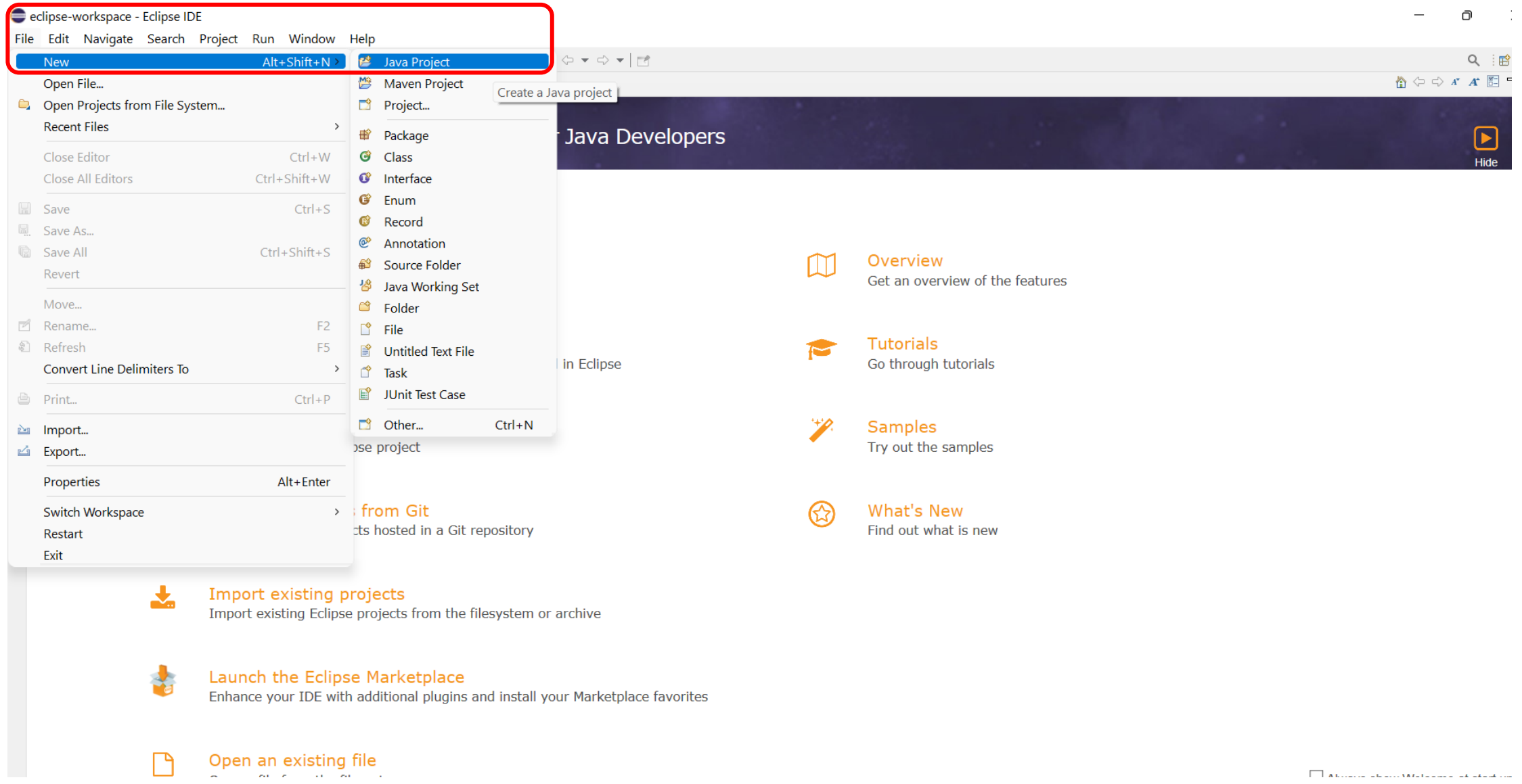
If you choose Eclipse IDE (Java)

- Install Eclipse Software <https://www.eclipse.org/downloads/>



If you choose Eclipse IDE (Java)

- Install Eclipse Software <https://www.eclipse.org/downloads/>



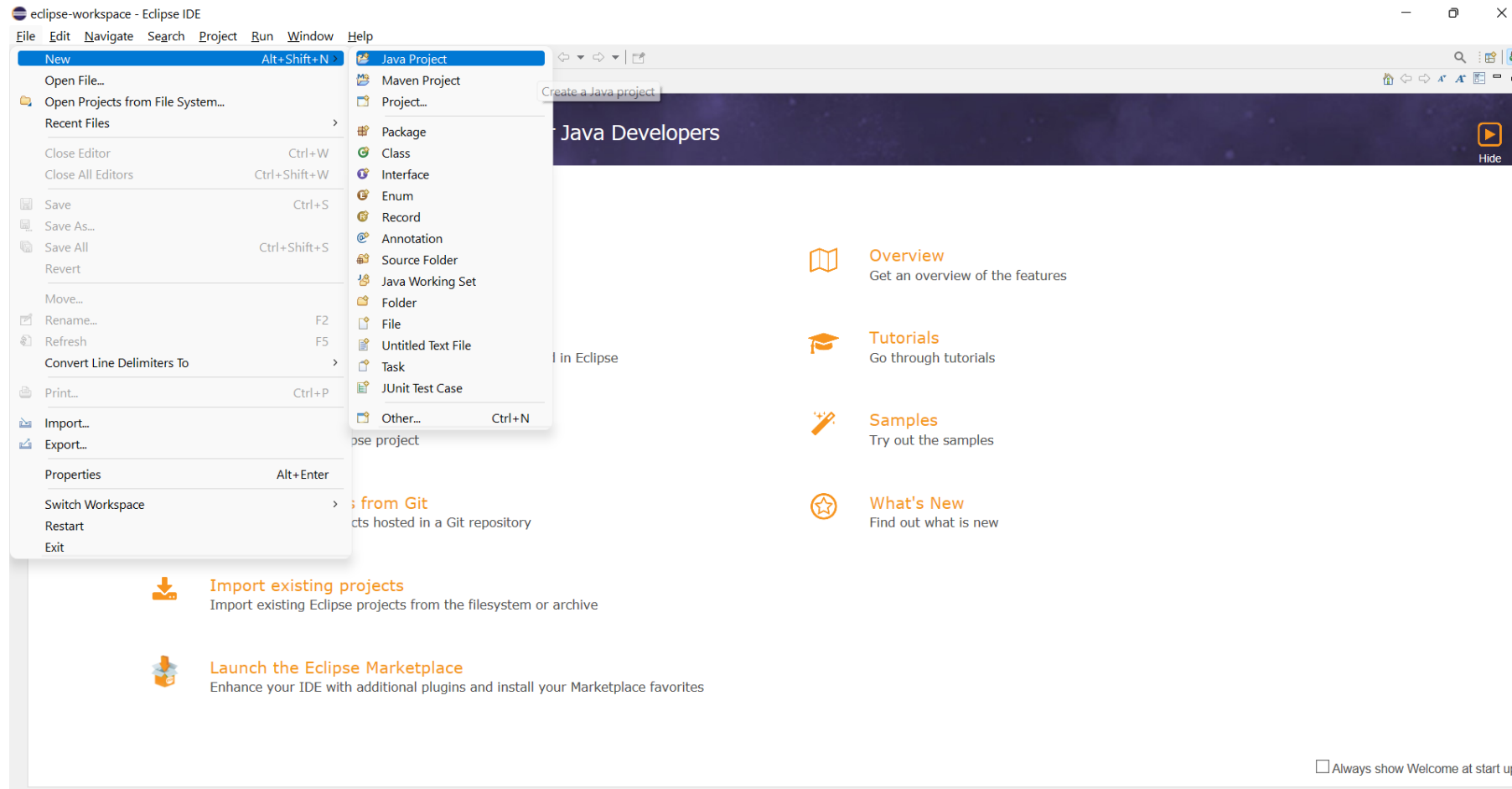
The First Java Program

We present a simple Java program (Listing 1.1) that prints "Welcome to Java!" to the console. We demonstrate the basic structure of a Java program, including the class definition and the main method, which is the entry point of any Java application.

Simple User Input and Output Analysis in Java

1. Click on "Create a new Java project":

- This will open a new window where you can set up your project.



Simple User Input and Output Analysis in Java

2. Set up the new project:

- Enter a project name (e.g., Welcome).
- Click Finish.

The screenshot shows the 'New Project' dialog in the Eclipse IDE. The 'Project name' field is empty. The 'Location' is set to 'C:\Users\gaming 15\workspace\UserInputOutputProject'. The 'JRE' section has 'Use an execution environment JRE' selected with 'JavaSE-22' chosen. The 'Project layout' section has 'Create separate folders for sources and class files' selected. The 'Working sets' section has 'Add project to working sets' unchecked. The 'Module' section has 'Create module-info.java file' checked, and the 'Module name' field is empty. The 'Generate comments' checkbox is checked. The 'Finish' button is highlighted with a red rectangle.

☒ Use default location
Location: C:\Users\gaming 15\workspace\UserInputOutputProject [Browse...](#)

JRE

☒ Use an execution environment JRE: JavaSE-22 [Configure JREs...](#)

☐ Use a project specific JRE: jdk-22

☐ Use default JRE 'jdk-22' and workspace compiler preferences

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets [New...](#)

Working sets: [Select...](#)

Module

☒ Create module-info.java file

Module name:

☒ Generate comments
module name will be "UserInputOutputProject" (if no module is specified, then project name will be used as module name)

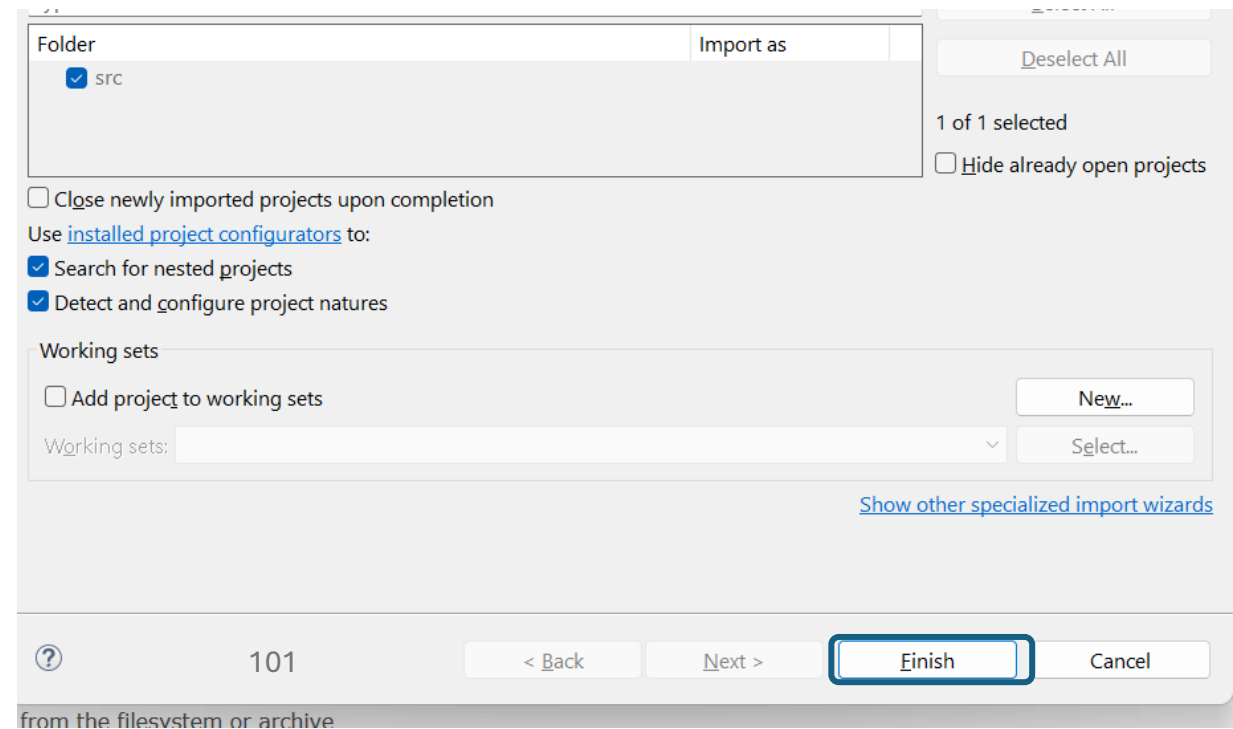
100

[? < Back](#) [Next >](#) **Finish** [Cancel](#)

Simple User Input and Output Analysis in Java

2. Set up the new project:

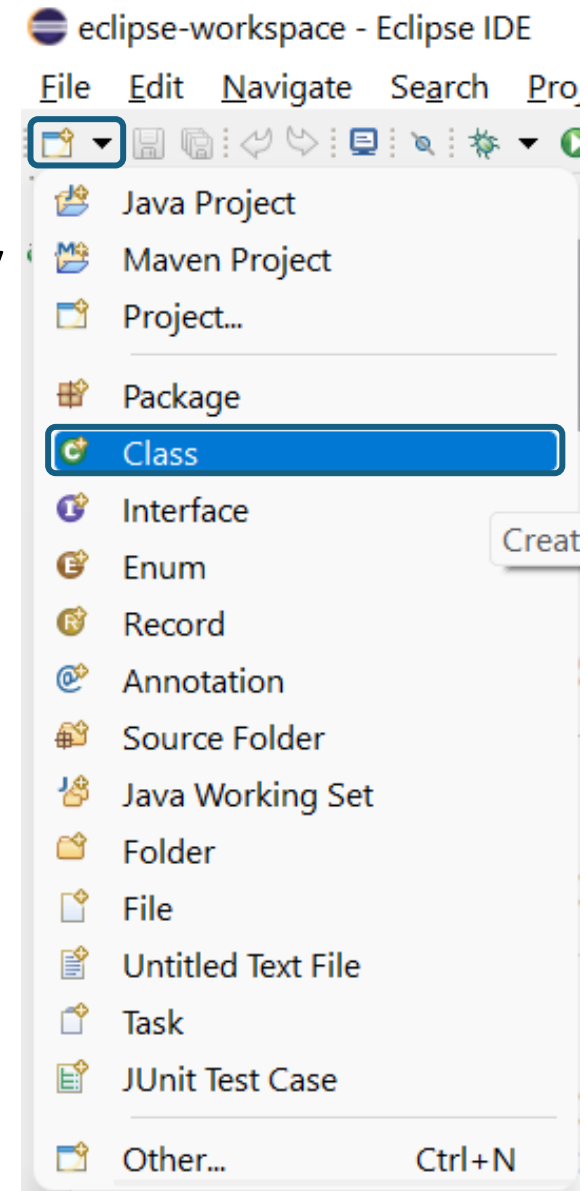
- Enter a project name (e.g., Welcome).
- Click Finish.



Simple User Input and Output Analysis in Java

3. Create a new Java class:

- File >> Open Projects from File Systems >> Directory >> Choose the Direct where the Project is Saved
'e.g. C:\Users\gaming 15\eclipse-workspace\Welcome\src'
- Click on Select Folder
- Select New > Class.



Simple User Input and Output Analysis in Java

- Enter a class name (e.g., Welcome).
- Ensure the public static void main(String[] args) option is checked.
- Click Finish.
- Click on Hide (Front Page)

The screenshot shows the 'New Class' dialog box in an IDE. The 'Enclosing type' checkbox is unchecked. The 'Name' field contains 'UserInputOutput'. Under 'Modifiers', 'public' is selected with a radio button, and 'abstract', 'final', and 'static' are unchecked. The 'Superclass' field contains 'java.lang.Object'. The 'Interfaces' field is empty. Under 'Which method stubs would you like to create?', 'public static void main(String[] args)' and 'Inherited abstract methods' are checked with checkboxes, while 'Constructors from superclass' is unchecked. The 'Do you want to add comments?' section has 'Generate comments' unchecked. At the bottom, there is a help icon, an 'Finish' button, and a 'Cancel' button.

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass:

Interfaces:

Which method stubs would you like to create?

☒ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Simple User Input and Output Analysis in Java

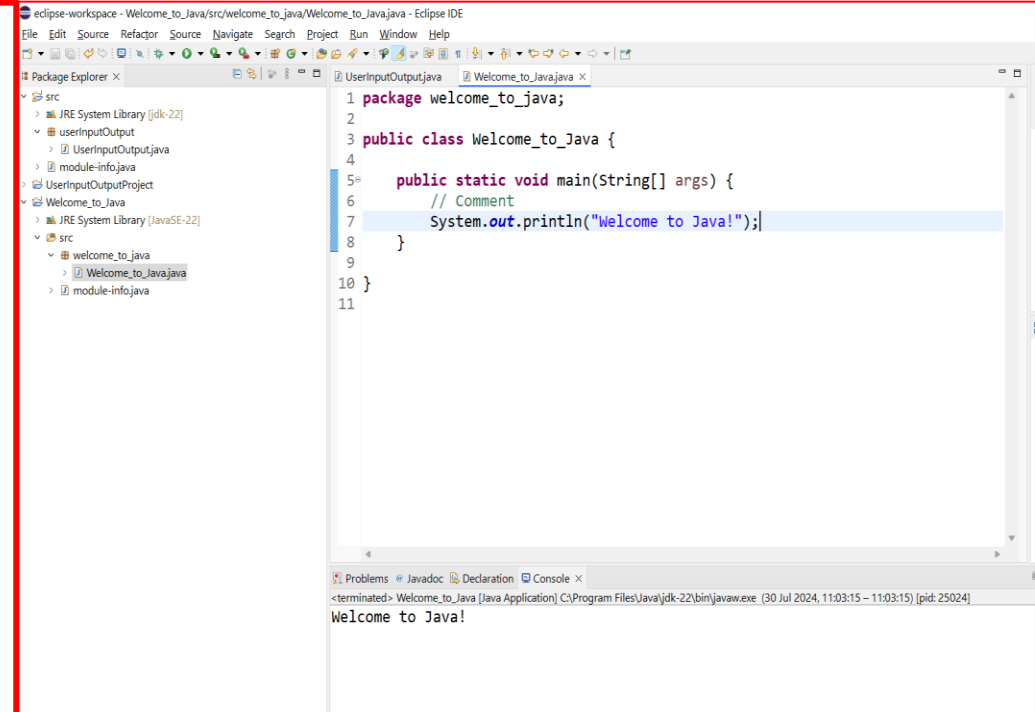
4. Write your Java code:

- Copy and paste the code into the newly created class:


```
package welcome_to_java;
```

```
public class Welcome_to_Java {
```

```
public static void main(String[]  
args) {  
    // Comment  
    System.out.println("Welcome to  
Java!");  
}  
}
```



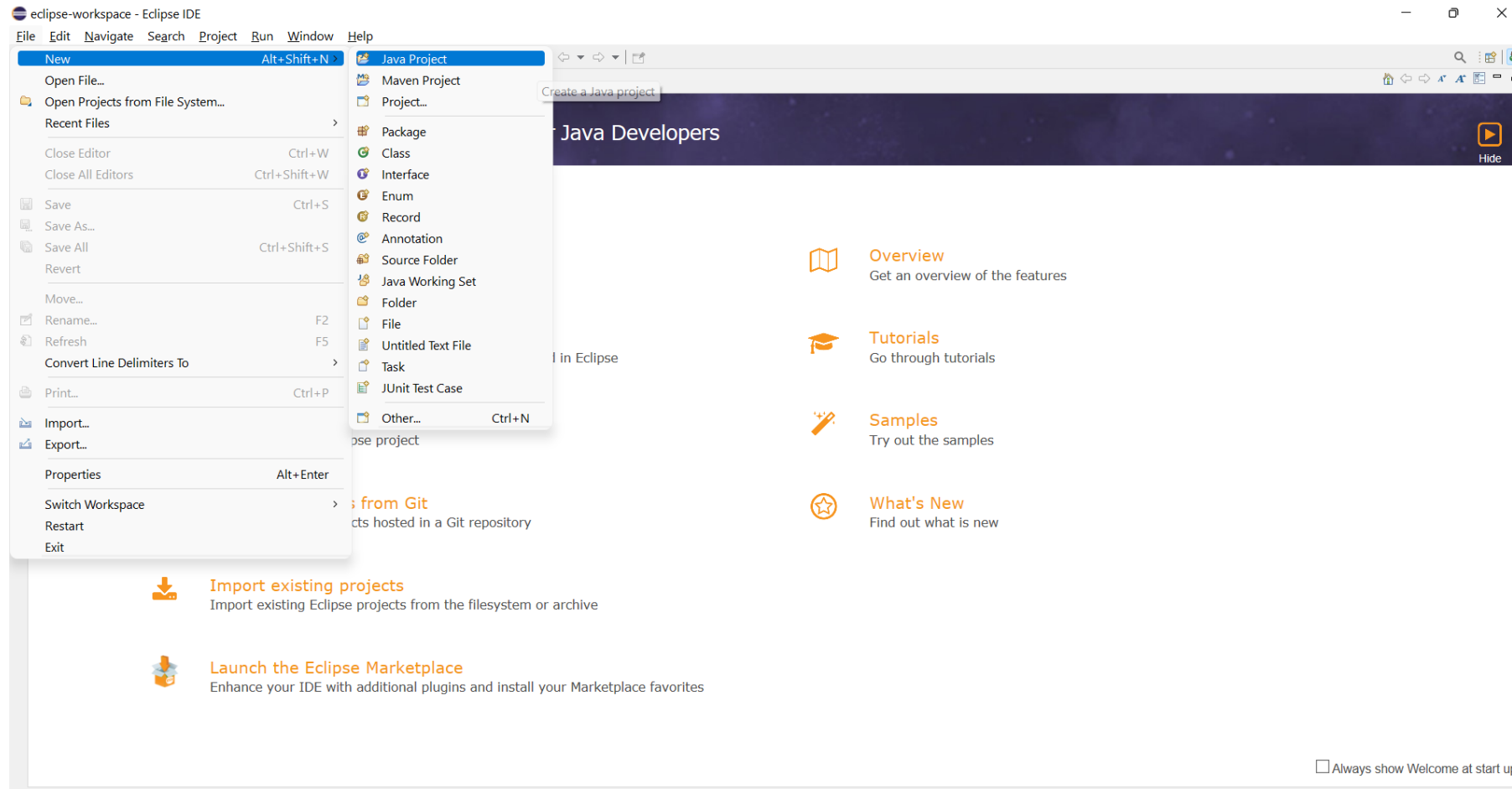
- public class Welcome: Defines a public class named Welcome.
- public static void main(String[] args): The main method is the entry point of the program.
- System.out.println("Welcome to Java!");: Prints the message "Welcome to Java!" to the console.

This simple program introduces you to the fundamental components of a Java program, such as classes, methods, and the standard output function. By running this program, you get a practical understanding of how Java code is structured and executed.

Simple User Input and Output Analysis in Java

1. Click on "Create a new Java project":

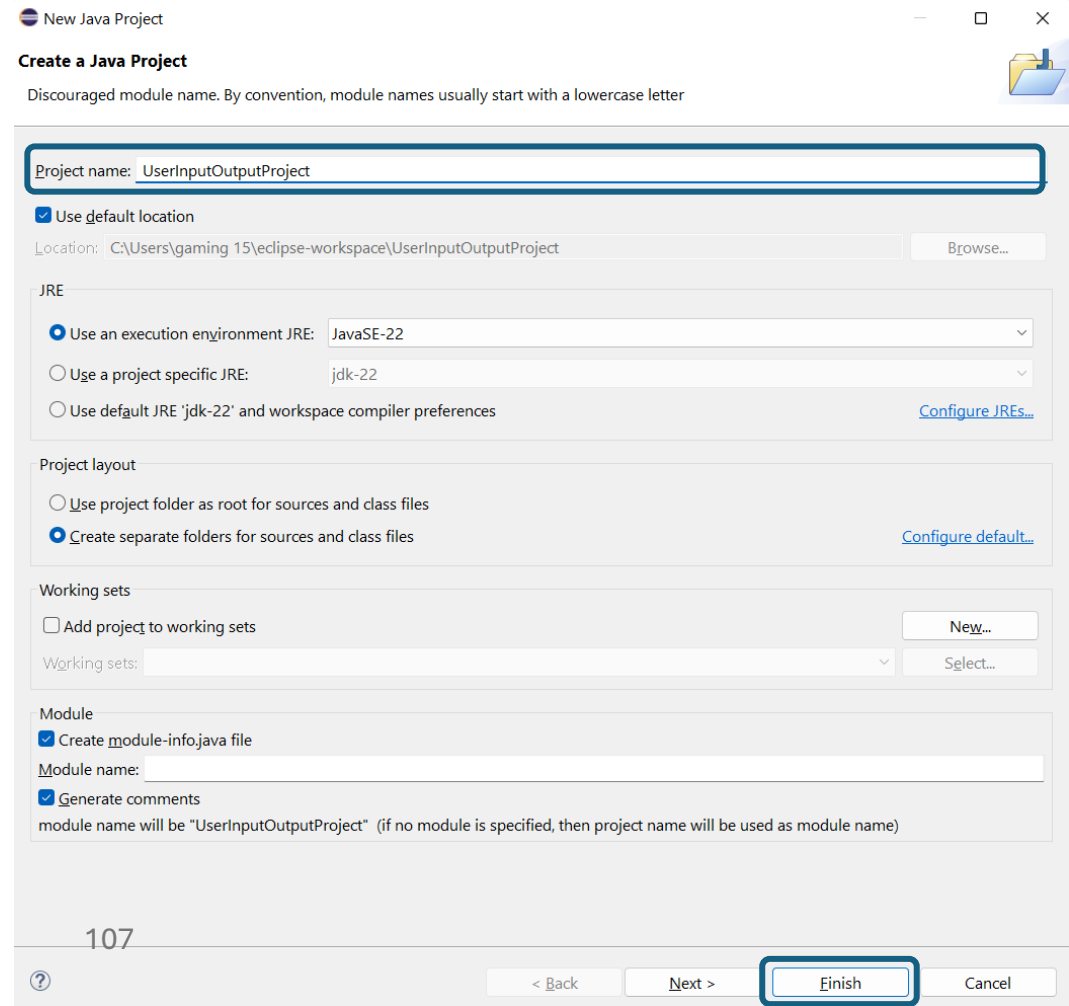
- This will open a new window where you can set up your project.



Simple User Input and Output Analysis in Java

2. Set up the new project:

- Enter a project name (e.g., UserInputOutputProject).
- Click Finish.



The screenshot shows the 'New Java Project' dialog box in the Eclipse IDE. The dialog is titled 'New Java Project' and has a subtitle 'Create a Java Project'. Below the subtitle, it says 'Discouraged module name. By convention, module names usually start with a lowercase letter'. The main form has several sections: 'Project name' with a text field containing 'UserInputOutputProject'; 'Use default location' with a checked checkbox and a 'Location' field showing 'C:\Users\gaming 15\eclipse-workspace\UserInputOutputProject'; 'JRE' section with three radio buttons: 'Use an execution environment JRE' (selected, showing 'JavaSE-22'), 'Use a project specific JRE' (showing 'jdk-22'), and 'Use default JRE 'jdk-22' and workspace compiler preferences'; 'Project layout' section with two radio buttons: 'Use project folder as root for sources and class files' and 'Create separate folders for sources and class files' (selected); 'Working sets' section with an unchecked checkbox 'Add project to working sets' and a 'Working sets' dropdown; 'Module' section with a checked checkbox 'Create module-info.java file', a 'Module name' field, and a checked checkbox 'Generate comments'. At the bottom, it says 'module name will be "UserInputOutputProject" (if no module is specified, then project name will be used as module name)'. The bottom of the dialog has navigation buttons: '< Back', 'Next >', 'Finish' (highlighted with a red box), and 'Cancel'. The page number '107' is visible in the bottom left corner.

New Java Project

Create a Java Project

Discouraged module name. By convention, module names usually start with a lowercase letter

Project name: UserInputOutputProject

☒ Use default location

Location: C:\Users\gaming 15\eclipse-workspace\UserInputOutputProject [Browse...](#)

JRE

☒ Use an execution environment JRE: JavaSE-22

☐ Use a project specific JRE: jdk-22

☐ Use default JRE 'jdk-22' and workspace compiler preferences [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets [New...](#)

Working sets: [Select...](#)

Module

☒ Create module-info.java file

Module name:

☒ Generate comments

module name will be "UserInputOutputProject" (if no module is specified, then project name will be used as module name)

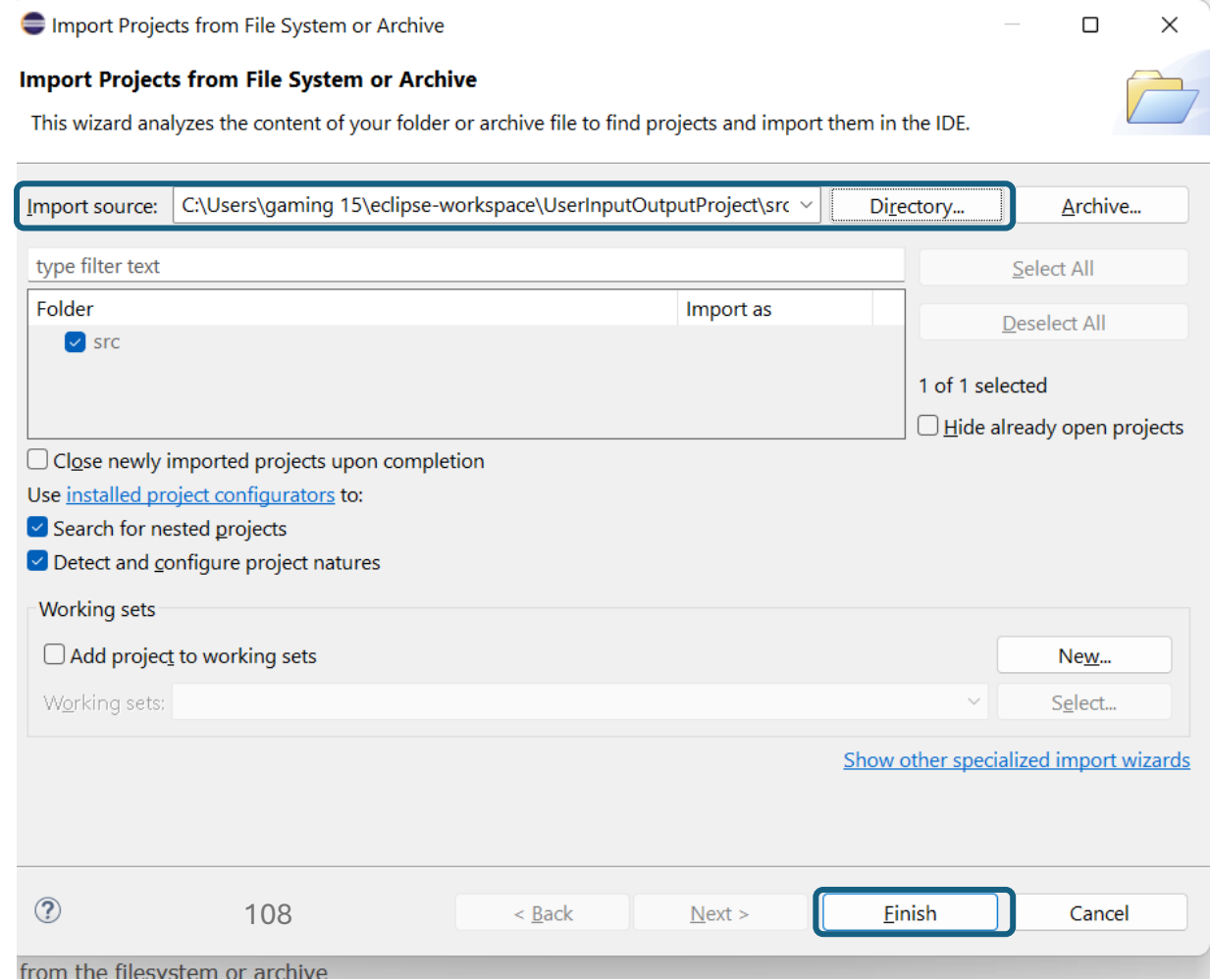
107

[?](#) < Back Next > Finish Cancel

Simple User Input and Output Analysis in Java

2. Set up the new project:

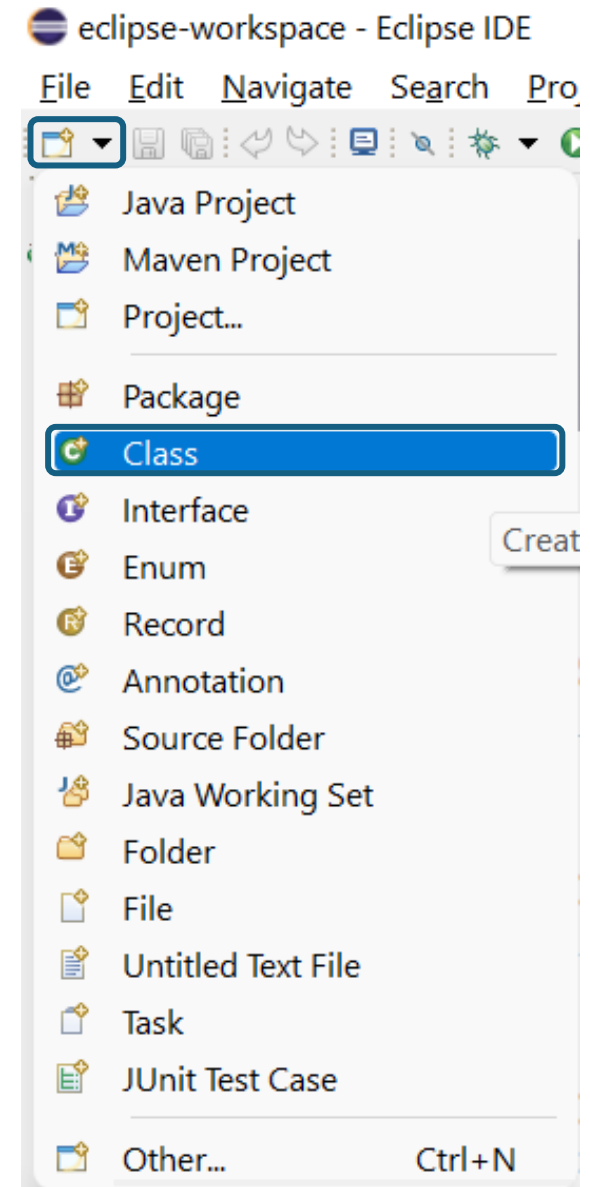
- Enter a project name (e.g., UserInputOutputProject).
- Click Finish.



Simple User Input and Output Analysis in Java

3. Create a new Java class:

- File >> Open Projects from File Systems >> Directory >> Choose the Direct where the Project is Saved 'e.g. C:\Users\gaming 15\eclipse-workspace\UserInputOutputProject\src'
- Click on Select Folder
- Select New > Class.



Simple User Input and Output Analysis in Java

- Enter a class name (e.g., UserInputOutput).
- Ensure the public static void main(String[] args) option is checked.
- Click Finish.
- Click on Hide (Front Page)

New Java Class

Java Class

Create a new Java class.

Source folder: src Browse...

Package: userInputOutput Browse...

☐ Enclosing type: Browse...

Name: UserInputOutput

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass: java.lang.Object Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?

☒ public static void main(String[] args)

☐ Constructors from superclass

☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))

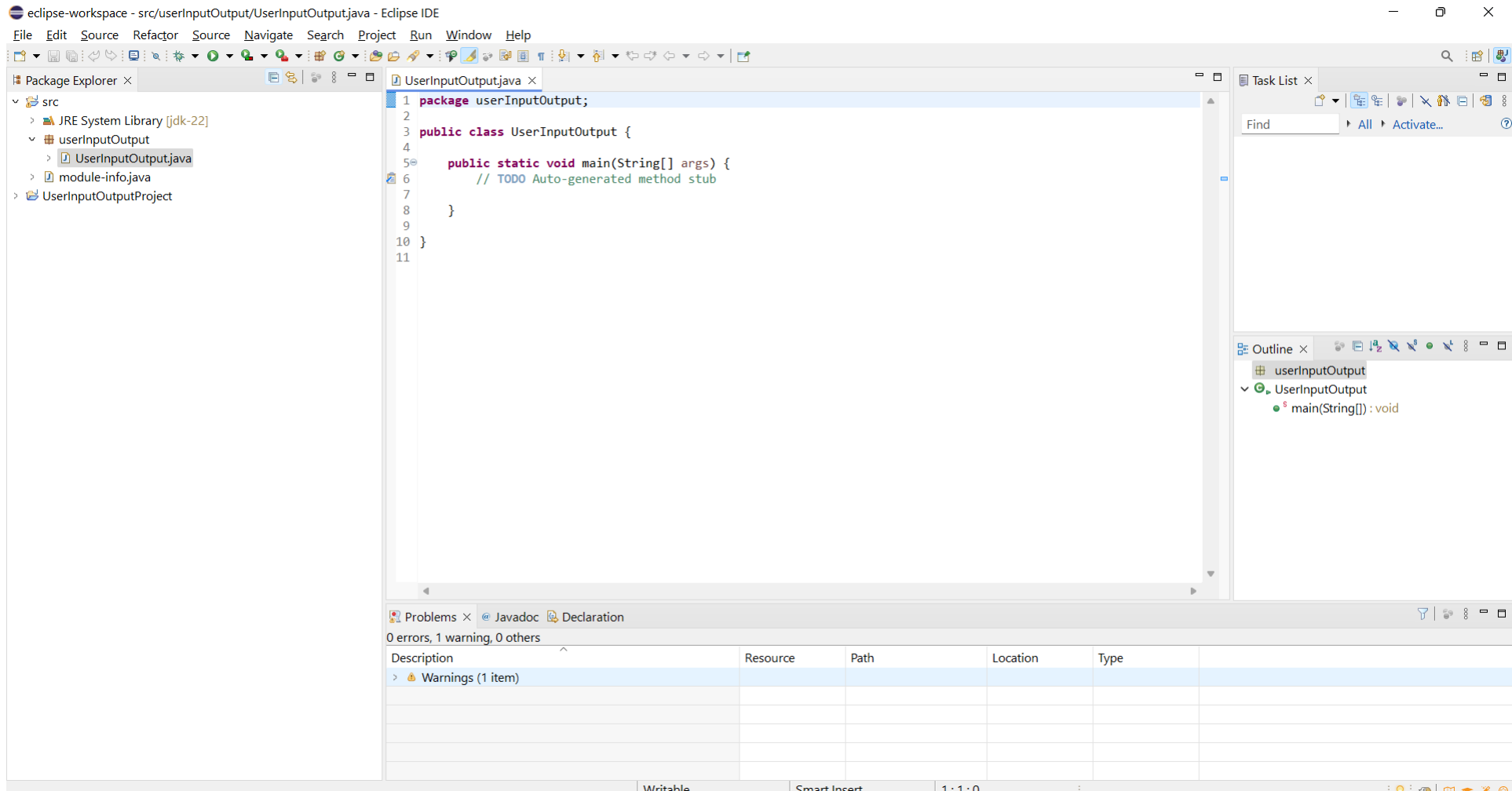
☐ Generate comments

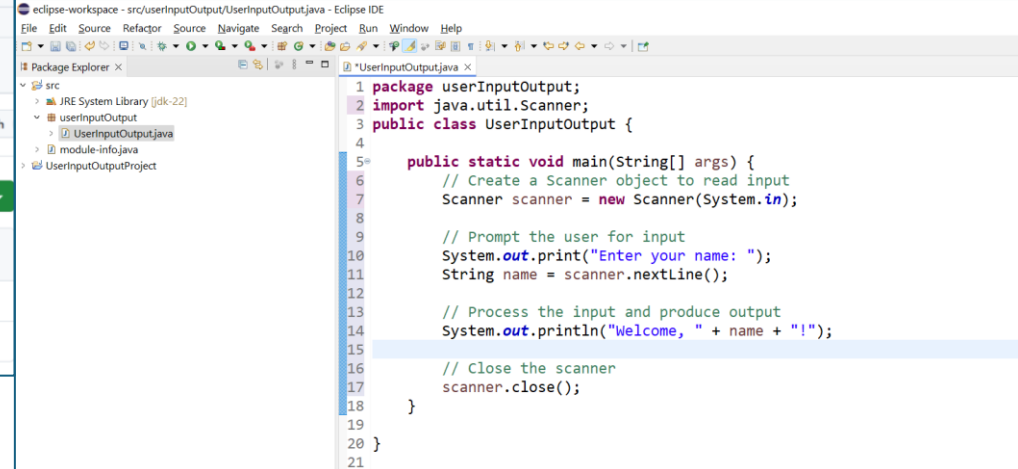
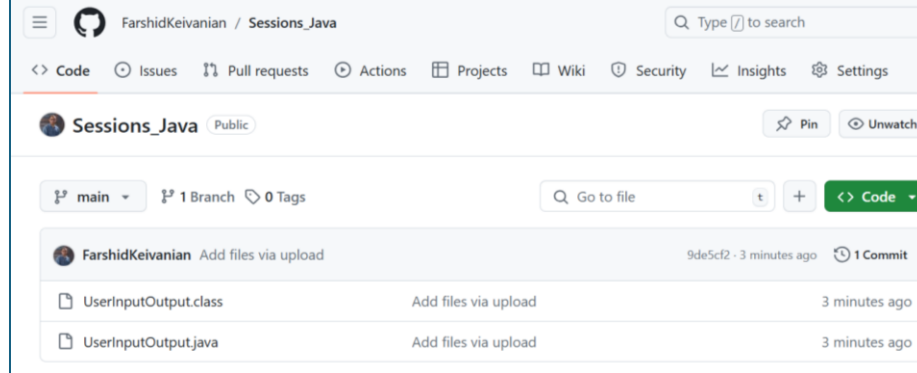
Finish Cancel

Simple User Input and Output Analysis in Java

4. Write your Java code:

- Copy and paste the code into the newly created class:





```
package userInputOutput;
import java.util.Scanner;
public class UserInputOutput {
public static void main(String[] args) {
// Create a Scanner object to read input
Scanner scanner = new Scanner(System.in);
// Prompt the user for input
System.out.print("Enter your name: ");
String name = scanner.nextLine();
// Process the input and produce output
System.out.println("Welcome, " + name + "!");
// Close the scanner
scanner.close();
}
}
```



```
package userInputOutput;
import java.util.Scanner;
public class UserInputOutput {
public static void main(String[] args) {
// Create a Scanner object to read input
Scanner scanner = new Scanner(System.in);
// Prompt the user for input
System.out.print("Enter your name: ");
String name = scanner.nextLine();
// Process the input and produce output
System.out.println("Welcome, " + name + "!");
// Close the scanner
scanner.close();
}
}
```

1. Import Statement:

- import java.util.Scanner;
- This line imports the Scanner class, which is used for reading input from the user.

```
package userInputOutput;
import java.util.Scanner;
public class UserInputOutput {
public static void main(String[] args) {
// Create a Scanner object to read input
Scanner scanner = new Scanner(System.in);
// Prompt the user for input
System.out.print("Enter your name: ");
String name = scanner.nextLine();
// Process the input and produce output
System.out.println("Welcome, " + name + "!");
// Close the scanner
scanner.close();
}
}
```

2. Class Declaration:

- public class UserInputOutput {
- This line declares a public class named UserInputOutput

```
package userInputOutput;
import java.util.Scanner;
public class UserInputOutput {
public static void main(String[] args) {
// Create a Scanner object to read input
Scanner scanner = new Scanner(System.in);
// Prompt the user for input
System.out.print("Enter your name: ");
String name = scanner.nextLine();
// Process the input and produce output
System.out.println("Welcome, " + name + "!");
// Close the scanner
scanner.close();
}
}
```

3. Main Method:

- public static void main(String[] args) {
- This is the entry point of the program where execution begins.

```
package userInputOutput;
import java.util.Scanner;
public class UserInputOutput {
public static void main(String[] args) {
// Create a Scanner object to read input
Scanner scanner = new Scanner(System.in);
// Prompt the user for input
System.out.print("Enter your name: ");
String name = scanner.nextLine();
// Process the input and produce output
System.out.println("Welcome, " + name + "!");
// Close the scanner
scanner.close();
}
}
```

4. Scanner Object Creation:

- Scanner scanner = new Scanner(System.in);
- This line creates a Scanner object to read input from the standard input stream (keyboard).

```
package userInputOutput;  
import java.util.Scanner;  
public class UserInputOutput {  
    public static void main(String[] args) {  
        // Create a Scanner object to read input  
        Scanner scanner = new Scanner(System.in);  
        // Prompt the user for input  
        System.out.print("Enter your name: ");  
        String name = scanner.nextLine();  
        // Process the input and produce output  
        System.out.println("Welcome, " + name + "!");  
        // Close the scanner  
        scanner.close();  
    }  
}
```

5. Prompting for Input:

- `System.out.print("Enter your name: ");`
- This line prompts the user to enter their name.

6. Reading Input:

- `String name = scanner.nextLine();`
- This line reads the user's input and stores it in a variable named `name`.

```
package userInputOutput;
import java.util.Scanner;
public class UserInputOutput {
public static void main(String[] args) {
// Create a Scanner object to read input
Scanner scanner = new Scanner(System.in);
// Prompt the user for input
System.out.print("Enter your name: ");
String name = scanner.nextLine();
// Process the input and produce output
System.out.println("Welcome, " + name + "!");
// Close the scanner
scanner.close();
}
}
```

7. Producing Output:

- `System.out.println("Welcome, " + name + "!");`
- This line outputs a welcome message that includes the user's name.

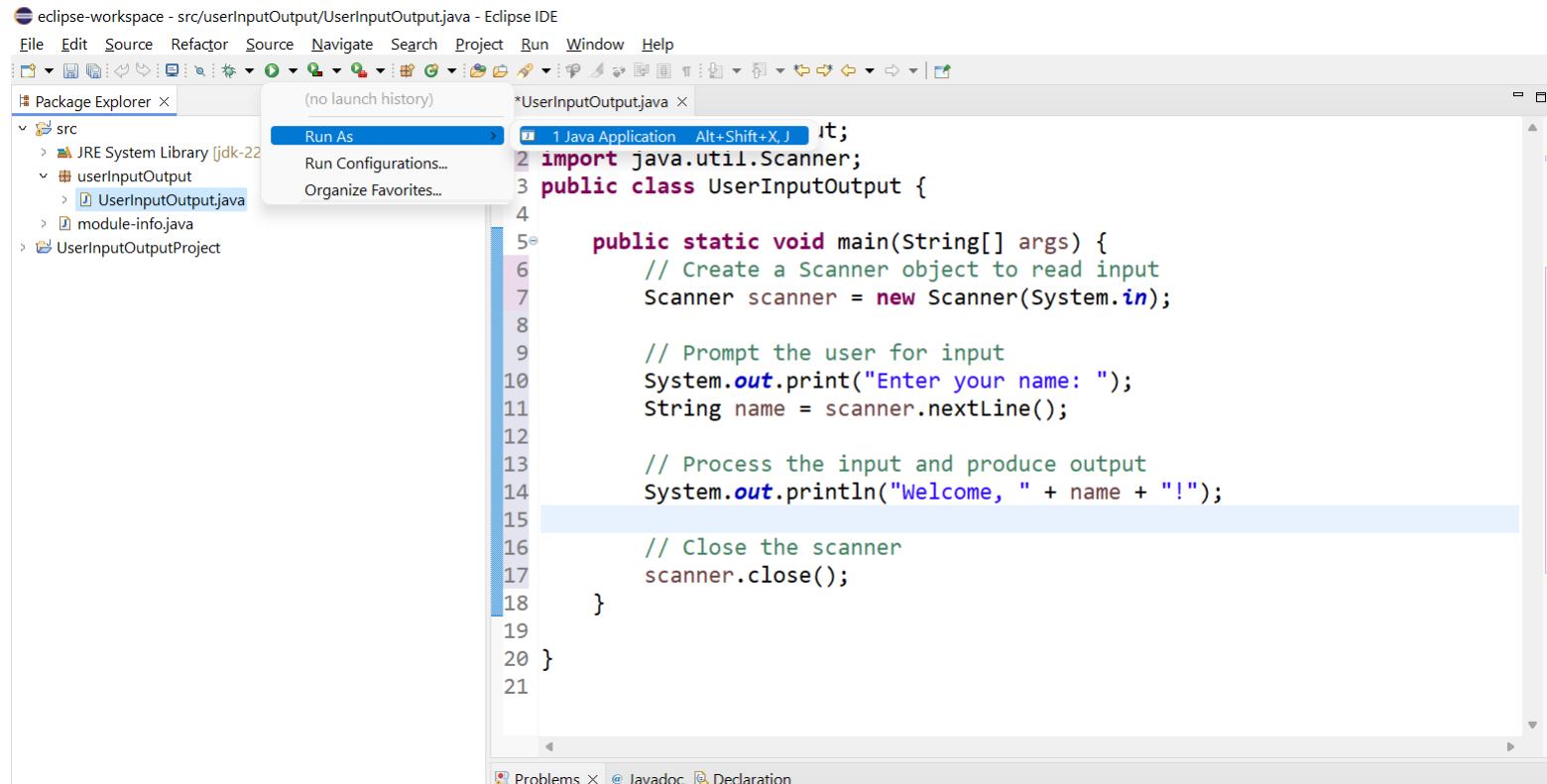
8. Closing the Scanner:

- `scanner.close();`
- This line closes the Scanner object to free up resources.

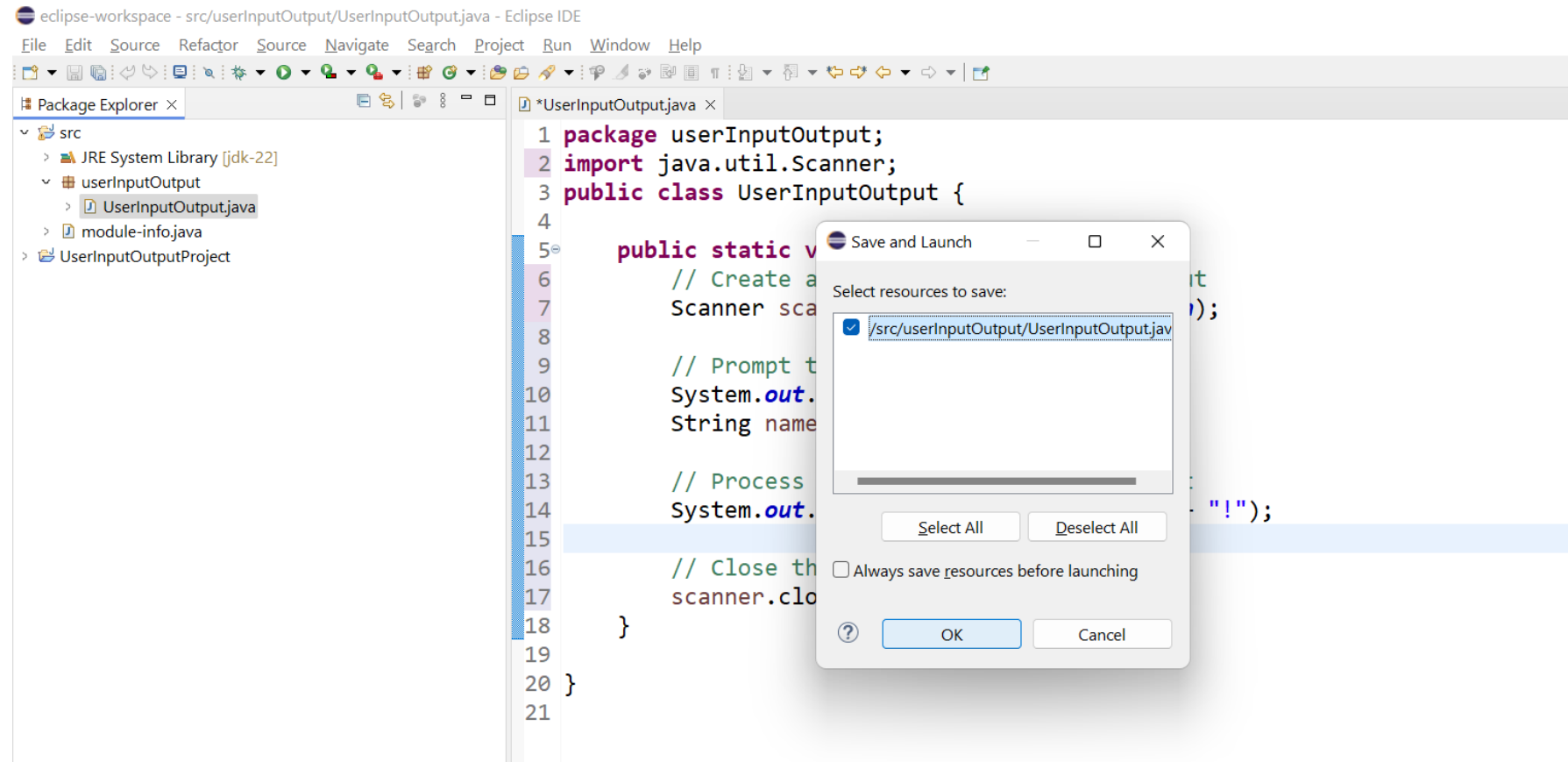
5. Run your Java program:

- Right-click on your Java file (e.g., UserInputOutput.java).
- Select Run As > Java Application.

This will execute your Java program, allowing you to enter your name and see the welcome message displayed in the console.



Select and Click on OK



Output

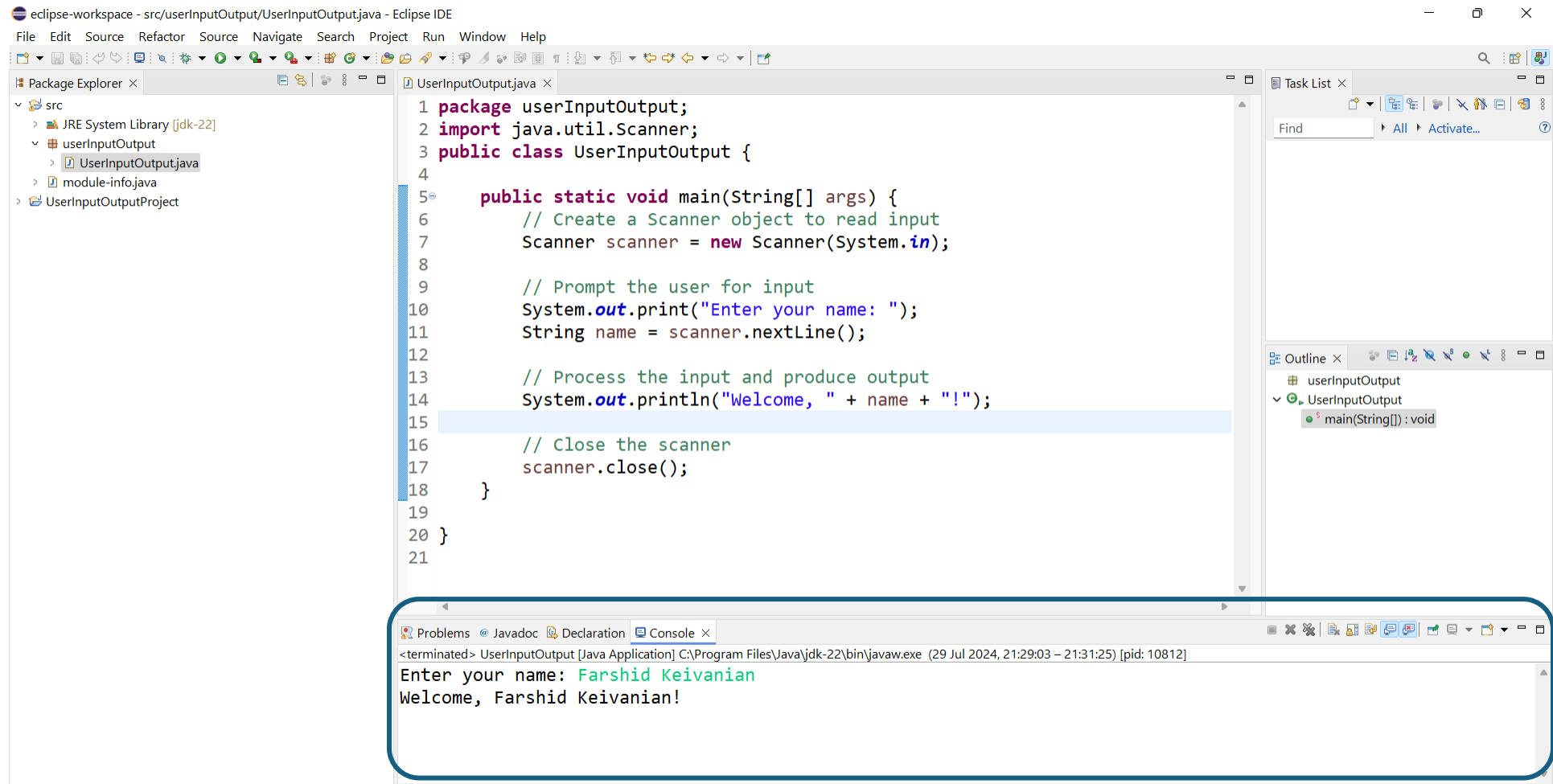
The screenshot displays the Eclipse IDE interface. The main editor window shows the source code for `UserInputOutput.java`. The code defines a package, imports `java.util.Scanner`, and creates a `UserInputOutput` class with a `main` method. The `main` method prompts the user for their name and prints a welcome message. The Package Explorer on the left shows the project structure. The Task List and Outline views are also visible. At the bottom, the Console view shows the output of the program, which is "Enter your name:". A blue rectangular box highlights the Console view.

```
1 package userInputOutput;
2 import java.util.Scanner;
3 public class UserInputOutput {
4
5     public static void main(String[] args) {
6         // Create a Scanner object to read input
7         Scanner scanner = new Scanner(System.in);
8
9         // Prompt the user for input
10        System.out.print("Enter your name: ");
11        String name = scanner.nextLine();
12
13        // Process the input and produce output
14        System.out.println("Welcome, " + name + "!");
15
16        // Close the scanner
17        scanner.close();
18    }
19 }
20 }
21 }
```

Console Output:

```
UserInputOutput [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe (29 Jul 2024, 21:29:03) [pid: 10812]
Enter your name:
```

Output



The screenshot displays the Eclipse IDE interface. The main editor window shows the source code for `UserInputOutput.java`. The code defines a `main` method that prompts the user for their name and prints a welcome message. The Package Explorer on the left shows the project structure. The Task List and Outline views are on the right. At the bottom, the Console view shows the program's execution output, which is highlighted with a blue rounded rectangle. The output shows the program terminating, followed by the prompt "Enter your name:" and the user input "Farshid Keivanian", resulting in the output "Welcome, Farshid Keivanian!".

```
1 package userInputOutput;
2 import java.util.Scanner;
3 public class UserInputOutput {
4
5     public static void main(String[] args) {
6         // Create a Scanner object to read input
7         Scanner scanner = new Scanner(System.in);
8
9         // Prompt the user for input
10        System.out.print("Enter your name: ");
11        String name = scanner.nextLine();
12
13        // Process the input and produce output
14        System.out.println("Welcome, " + name + "!");
15
16        // Close the scanner
17        scanner.close();
18    }
19
20 }
21
```

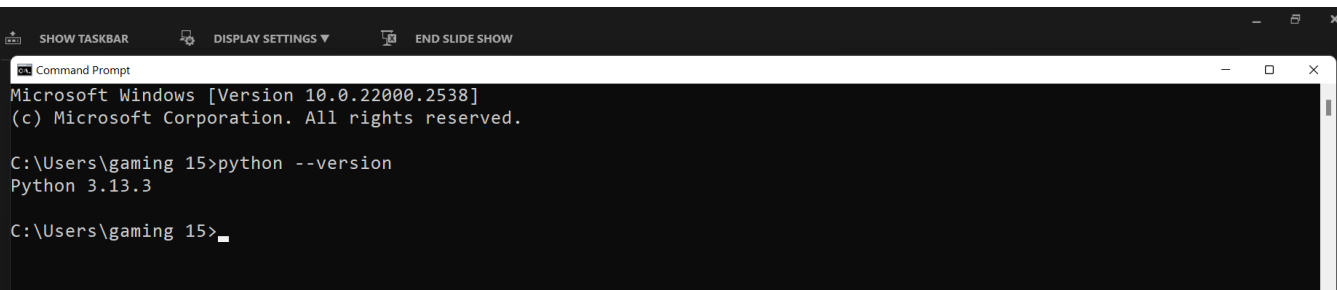
<terminated> UserInputOutput [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe (29 Jul 2024, 21:29:03 – 21:31:25) [pid: 10812]
Enter your name: Farshid Keivanian
Welcome, Farshid Keivanian!

If you choose VS Code (Python)

Step 1: Install Python

For Windows 11:

1. Go to the official site: <https://www.python.org/downloads/>
2. Download the latest Python 3.x Windows Installer.
3. Important: During installation, check the box that says Add Python 3.x to PATH before clicking Install Now.
4. After installation, open Command Prompt (cmd) and type: `python --version`



```
Microsoft Windows [Version 10.0.22000.2538]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gaming 15>python --version
Python 3.13.3

C:\Users\gaming 15>_
```

You should see the Python version printed.

This confirms it's correctly installed and added to PATH.

If you choose VS Code (Python)

For macOS:

1. Download the macOS installer from

<https://www.python.org/downloads/mac-osx/>

2. Follow the installer instructions.

3. To verify, open Terminal and run:

```
python3 --version
```

(On Mac, we often use python3 instead of python.)

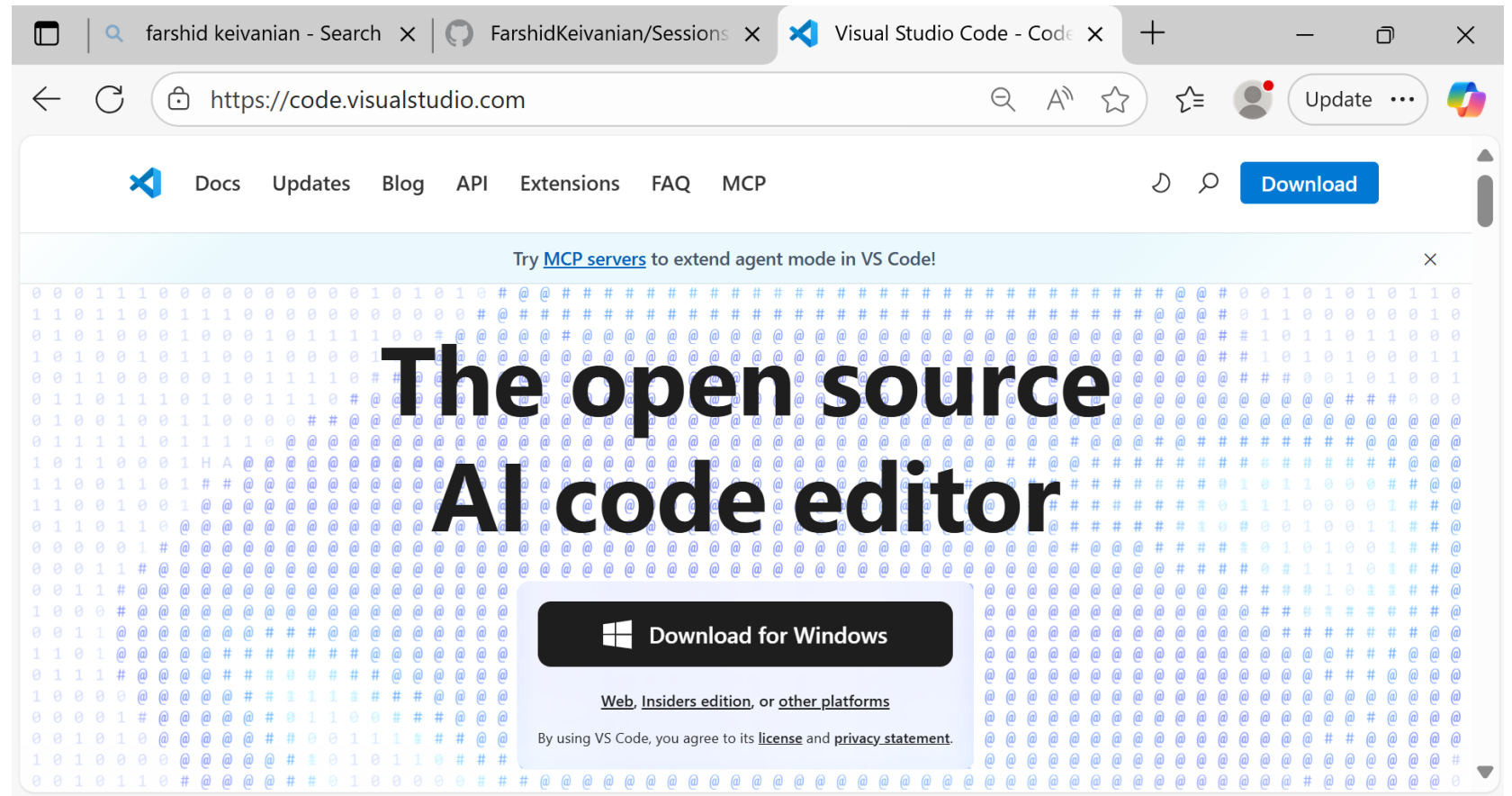
To update:

```
python -m pip install --upgrade pip
```

If you choose VS Code (Python)

Step 2: Install VS Code

1. Download VS Code: <https://code.visualstudio.com/>
2. Install it like any other application.
3. Open VS Code.



If you choose VS Code (Python)

Search and Install:

1. Search: Python
 - Install the Python extension by Microsoft (most important!).
2. (Optional but helpful) Install:
 - Jupyter (for .ipynb interactive notebook support)
 - Pylance (for IntelliSense)
 - Code Runner (if students want simpler one-click run for .py files)

If you choose VS Code (Python)

Step 4: Enable the Run Button (play) with Interactive Window

1. Open a .py file or create a new file and save it as yourfile.py.
2. After the Python extension is installed, VS Code will show a play Run button in the top right.
3. Click the dropdown arrow next to the play and choose:
 - "Run Current File in Interactive Window".

If you are using a .ipynb (Jupyter Notebook) file, it will automatically show “Run Cell” and work interactively once the Jupyter extension is installed.

If you choose VS Code (Python)

Step 5: Select Python Interpreter (If Not Detected)

1. Press Ctrl+Shift+P (or Cmd+Shift+P on Mac) to open the command palette.
2. Type: Python: Select Interpreter
3. Choose the one that shows your installed Python path (e.g., Python 3.x with correct version).

If you choose VS Code (Python)

Troubleshooting Common Issues

Run Button Not Appearing?

- Make sure:
 - Python extension is installed.
 - File is saved with .py or .ipynb extension.
 - Python interpreter is selected in bottom-left/status bar.
- **Getting “Python not found”?**
- On Windows, make sure Add to PATH was checked during installation.
- On macOS, use python3 instead of python.

If you choose VS Code (Python)

Summary Checklist

Step	Task	Windows	Mac
1	Install Python	[Add to PATH]	[Use python3]
2	Install VS Code	✓	✓
3	Install Extensions	Python, Jupyter	Same
4	Configure Interpreter	Ctrl+Shift+P → Select Python	Same
5	Run Code	Play → Interactive Window	Play works the same

If you choose VS Code (Python)

Sometimes, students may see an error like:

"Cannot run cells, ipykernel is not installed."

or

"Kernel 'Python 3' cannot be started using the selected interpreter."

This happens if:

- They installed Python but haven't installed any packages (especially in a new/clean environment).
- They're using a virtual environment that doesn't have ipykernel.

If you choose VS Code (Python)

To fix it (if needed), they can run:

On Windows (Command Prompt):

```
pip install ipykernel
```

If you choose VS Code (Python)

On macOS (Terminal):

```
pip3 install ipykernel
```

If you choose VS Code (Python)

Or from inside VS Code's terminal (Ctrl + ~), run:

```
python -m pip install ipykernel
```

How to Succeed in the Unit

Pro Tips:

- Pick one language (Java or Python) and stick with it
- Use UML tools to map your ideas before coding
- Use Git for version control from the start
- Reach out if you're stuck — don't wait until it's too late!

UML Diagrams

- Draw.io is Suitable:

Free and browser-based — no installation needed

Supports all standard UML diagram types: use case, activity, class, sequence, etc.

Can save files to local storage or Google Drive

Easy to export as .png, .pdf, or .drawio files for submission

UML Diagrams

UML Tool Recommendation:

You can use [Draw.io \(diagrams.net\)](https://draw.io) to create UML diagrams like Use Case, Activity, or Class Diagrams. Export your diagrams as .png or .drawio and upload with your lab submission.

Recommended Software Configuration Management (SCM): Git & Git Bash

Why Use Git?

- Git is a fast, distributed version control system ideal for tracking code changes and collaborating with others.
- It's widely used in industry (e.g., GitHub, GitLab, Bitbucket).

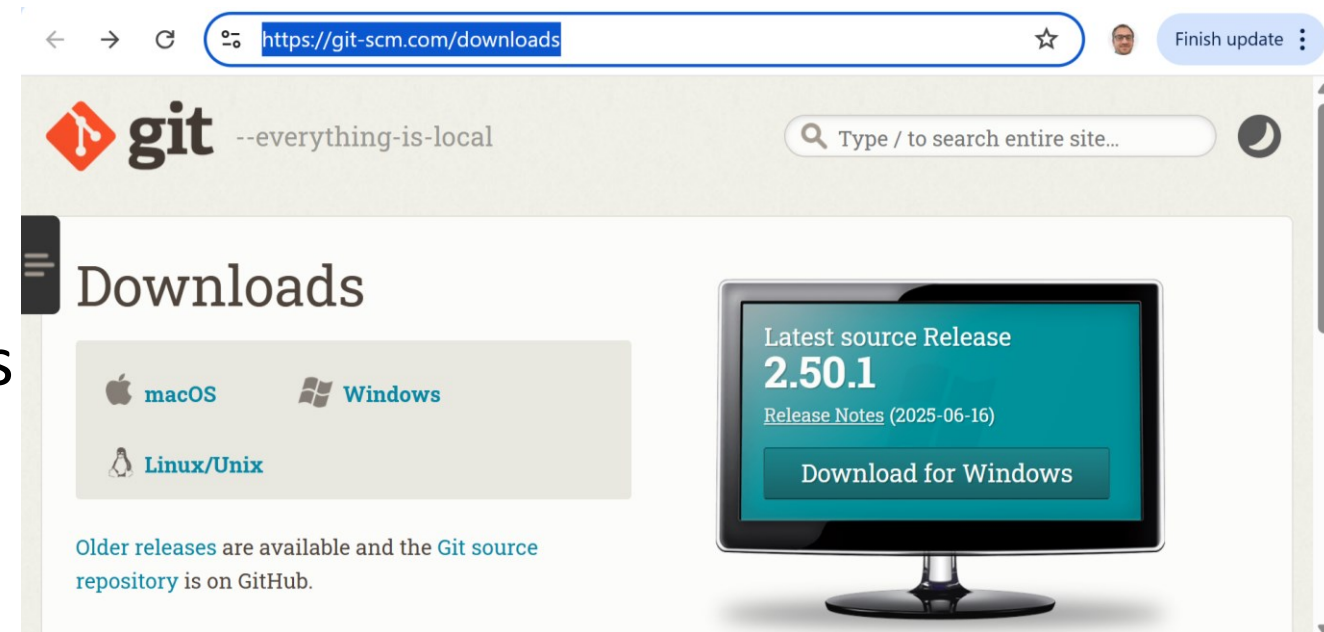
Recommended Software Configuration Management (SCM): Git & Git Bash

Recommended Tools:

- **Git Bash (Windows):** A command-line interface that provides Unix-style commands to use Git on Windows easily.
- **Git for Windows** (latest: v2.50.1, released 8 July 2025):

- [Downloads](#)
- [Download Git for Windows \(x64\)](#)
- Install using terminal (if Winget is available):

winget install --id Git.Git -e --source winget



Modern Software Development Tools (2026–2028)

What do software developers use in industry?

Category	Common Tools
Programming Languages	Java, Python, JavaScript, TypeScript, C#, Go
IDEs	VS Code, IntelliJ IDEA, Eclipse, PyCharm, Visual Studio
Version Control	Git, GitHub, GitHub Desktop, GitLab
Project Management	Jira, Confluence, Azure DevOps, Trello
UML & Design	diagrams.net (Draw.io), Lucidchart
Databases	MySQL, PostgreSQL, SQL Server, MongoDB
Cloud Platforms	Microsoft Azure, AWS, Google Cloud
Containers	Docker
Testing	JUnit, pytest, Selenium
CI/CD	GitHub Actions, Azure DevOps, Jenkins
Communication	Microsoft Teams, Slack

Modern Software Development Tools (2026–2028)

AI Tools Used by Software Developers (2026–2028)

Artificial Intelligence is becoming a standard part of software development.

Developers commonly use AI to:

- Generate code
- Explain code
- Debug programs
- Write documentation
- Create unit tests
- Improve code quality
- Learn new programming concepts

Modern Software Development Tools (2026–2028)

Popular AI Tools

AI Tool	Common Use
GitHub Copilot	AI coding assistant inside the IDE
ChatGPT	Code explanations, debugging, learning, documentation
Microsoft Copilot	Programming, Microsoft 365, Azure development
Gemini	Programming assistance and code explanation
Claude	Software design, documentation, code review
Amazon Q Developer	AWS development and cloud programming
Cursor AI	AI-powered code editor
JetBrains AI Assistant	AI integrated into IntelliJ IDEA

Modern Software Development Tools (2026–2028)

AI does not replace software developers.

Developers still need to:

- Understand the problem
- Design the solution
- Verify AI-generated code
- Test the software
- Ensure security and privacy
- Communicate with users and team members

Your understanding is more important than AI-generated code.

Skills Employers Look For (2026–2028)

Technical Skills

- Programming (Java, Python)
- Object-Oriented Programming (OOP)
- SQL and Databases
- Git and GitHub
- Cloud Computing (Azure or AWS)
- Software Testing
- APIs
- Problem Solving
- AI-assisted Software Development

Skills Employers Look For (2026–2028)

Professional Skills

- Communication
- Teamwork
- Time Management
- Critical Thinking
- Continuous Learning
- Ethical and Responsible AI Use

Suggested Learning Roadmap

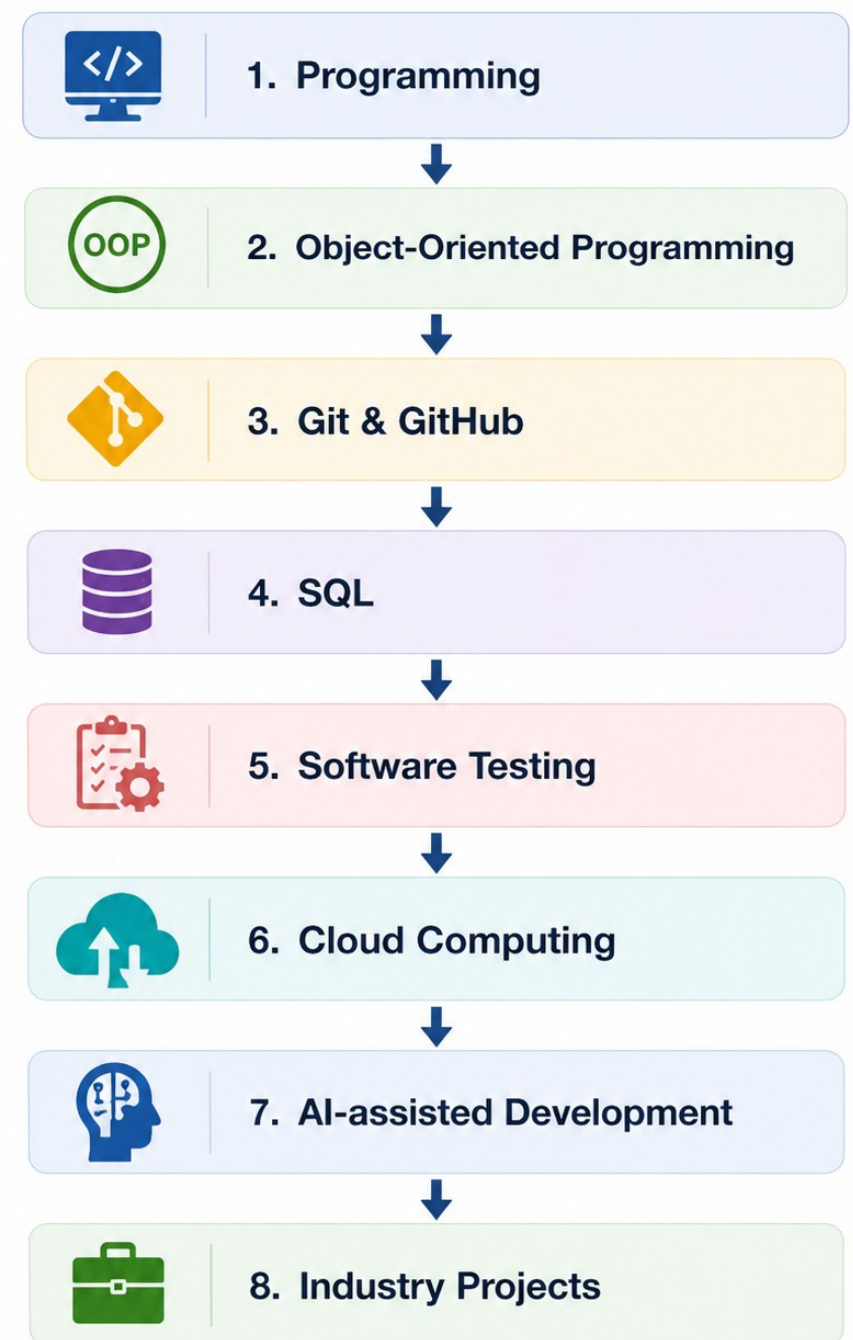
Don't try to learn everything at once.

Instead, become confident in:

- ✓ Java or Python
- ✓ GitHub
- ✓ Object-Oriented Programming
- ✓ UML
- ✓ SQL

Then gradually build additional skills in:

- Cloud Computing
- Artificial Intelligence
- DevOps
- Cybersecurity



Suggested Learning Roadmap

Companies are not looking for developers who can memorise syntax. They are looking for developers who can analyse problems, design solutions, collaborate with others, use modern tools—including AI—responsibly, and continue learning throughout their careers.

Week 1 Summary and Key Takeaways

This Week You Learned

Software Development Fundamentals

- What software development is
- The Software Development Life Cycle (SDLC)
- Waterfall and Agile methodologies

Software Modelling

- Introduction to UML (Unified Modeling Language)
- Requirements analysis
- Use cases and actors

Week 1 Summary and Key Takeaways

Programming Concepts

- Introduction to Object-Oriented Programming (OOP)
- Difference between CLI and GUI applications
- Java and Python development environments

Development Tools

- Canvas
- GitHub and GitHub Desktop
- VS Code, IntelliJ IDEA, Eclipse, or PyCharm
- diagrams.net (Draw.io)

Week 1 Summary and Key Takeaways

Assessments

- Assessment 1 (Case Study) – 70%
- Quiz 1 – 15%
- Quiz 2 – 15%

Week 1 Summary and Key Takeaways

Remember

- ✓ Programming is learned by practising, not just reading.
- ✓ Make mistakes—they are part of learning.
- ✓ Ask questions during tutorials.
- ✓ Start assessments early.
- ✓ Work consistently each week rather than leaving everything until the deadline.

Lab 1 Exercises

Complete the Lab 1 Exercises exercises as a team
(form groups of 3-4 students, one student acts as the client)

Lab 1 Exercises

Lab Tools:

The following tools are recommended to complete Lab 1 Exercises activities:

- Download and use [Visual Paradigm Community Edition](#)
- Use the online drawing tool: [draw.io](#)

Feel free to use other tools of choice!

Lab 1 Exercises

Lab Case Study: The bank system

Your team is hired to develop a desktop banking application for local bank. The bank manager has general IT knowledge but does not know programming too well.

The bank currently operates a web-based application. The objective is to have an in-branch desktop application that includes two types of banking interactions: CLI and GUI.

Both interfaces should be accessible using the branch computers where the application is installed. After consultation with the bank manager, the decision was to name the new application: DeskBankApp.

DeskBankApp enables registered bank customers and bank employees (Manager) to access the system.

DeskBankApp computers are mounted on top of small ATMs to allow customers make deposits or withdraws.

Lab 1 Exercises

Lab Case Study: **The bank system – The manager's requirements**

The bank manager requires the following functionalities in DeskBankApp:

- Customer can log in to the system (assume either GUI or CLI)
- Manager can log in to the system (assume either GUI or CLI)
- Customer can view their bank accounts
- Customer can make a deposit
- Customer can pay (an account or a credit card)
- Customer can withdraw
- Manager can view all customers
- Manager can add/remove a customer

Lab 1 Exercises

1. Read and understand the requirements, discuss the requirements within your team.
2. Perform requirements validation with stakeholders (client, tutor – as the delivery manager).
[check for: compliance, correctness, completeness, consistency]
3. Identify use cases.
4. Map use cases into a use case backlog.
5. Draw a use case diagram for the new DeskBankApp system.

Lab 1 Exercises

From Requirements to Use Case Diagram:

1. Gather and analyze requirements
2. Identify external actors
3. Define use cases based on user goals; focus on verbs
4. Draw system boundary
5. Connect actors to use cases
6. Identify relationships and denote them on the diagram

Lab 1 Exercises

Focus on the what not the how:

1. Some use cases can simply be mapped to functions
2. Some use cases are high-level and involve multiple steps or actions – and it also doesn't mean that each step corresponds to a function, e.g., a Marketplace application relies on a 3rd party payment gateway integration through APIs.

Lab 1 Exercises

Question 1

What is the name of the new banking application?

The application is called DeskBankApp.

Question 2

Why is DeskBankApp being developed?

The bank already has a web-based system, but it also wants an in-branch desktop application for branch computers.

Lab 1 Exercises

Question 3

What two types of interfaces must DeskBankApp provide?

DeskBankApp must provide:

- CLI — Command-Line Interface
- GUI — Graphical User Interface

Lab 1 Exercises

Question 4

Who can use DeskBankApp?

There are two main users:

- Customer
- Manager

These users are called actors in a use-case diagram.

Lab 1 Exercises

Question 5

What can a customer do?

A customer can:

- Log in
- View bank accounts
- Make a deposit
- Make a payment
- Withdraw money

Lab 1 Exercises

Question 6

What can a manager do?

A manager can:

- Log in
- View all customers
- Add a customer
- Remove a customer

Lab 1 Exercises

Question 7

What is requirements validation?

Requirements validation means checking whether the requirements are suitable before design and coding begin.

Check for:

- **Compliance** — Do the requirements follow relevant rules?
- **Correctness** — Do they describe what the bank actually needs?
- **Completeness** — Is anything important missing?
- **Consistency** — Do any requirements contradict each other?

Question 8

What questions should we ask the bank manager?

- What information is required for customer login?
- Can a customer have more than one account?
- What types of accounts are supported?
- Are there withdrawal limits?
- Can a customer pay another bank account?
- How is a credit-card payment processed?
- Can a manager edit customer information?
- What happens after several failed login attempts?
- Should transactions be stored in a file or database?
- Does the system need to print receipts?

These questions help make the requirements more complete.

Lab 1 Exercises

Question 9

What is an actor?

An actor is a person, device, or external system that interacts with the application.

For DeskBankApp:

- **Customer** is an actor.
- **Manager** is an actor.

Lab 1 Exercises

Question 10

What is a use case?

A use case describes a goal that an actor wants to achieve using the system.

Examples:

- Log in
- View accounts
- Make deposit
- Withdraw money
- Add customer

Use cases are normally written using verbs because they describe actions.

Question 11) Why should we focus on “what” rather than “how”?

A use-case diagram shows what the system must do, not the programming details.

For example:

Make Payment

is a suitable use case.

The diagram does not need to show:

- Python functions
- Java methods
- SQL queries
- API code
- Button-click code

Those details are considered later during design and development.

Lab 1 Exercises



Case Study

Your team is hired to develop a desktop banking application for a local bank. The application (DeskBankApp) provides both CLI and GUI interactions for customers and a GUI for managers. The system is installed on branch computers and mounted on top of small ATMs.



Manager's Requirements

- Customer can log in (GUI or CLI)
- Manager can log in (GUI or CLI)
- Customer can view their bank accounts
- Customer can make a deposit
- Customer can pay (an account or a credit card)
- Customer can withdraw
- Manager can view all customers
- Manager can add/remove a customer



Key Reminders

- Focus on the *what* not the *how*.
Use cases describe what users want to achieve, not how the system does it.

Lab 1 Exercises



From Requirements to Use Case Diagram

- 1 Gather and analyze requirements
- 2 Identify external actors
- 3 Define use cases based on user goals (focus on verbs)
- 4 Draw system boundary
- 5 Connect actors to use cases
- 6 Identify relationships and denote them on the diagram



Use Case Backlog

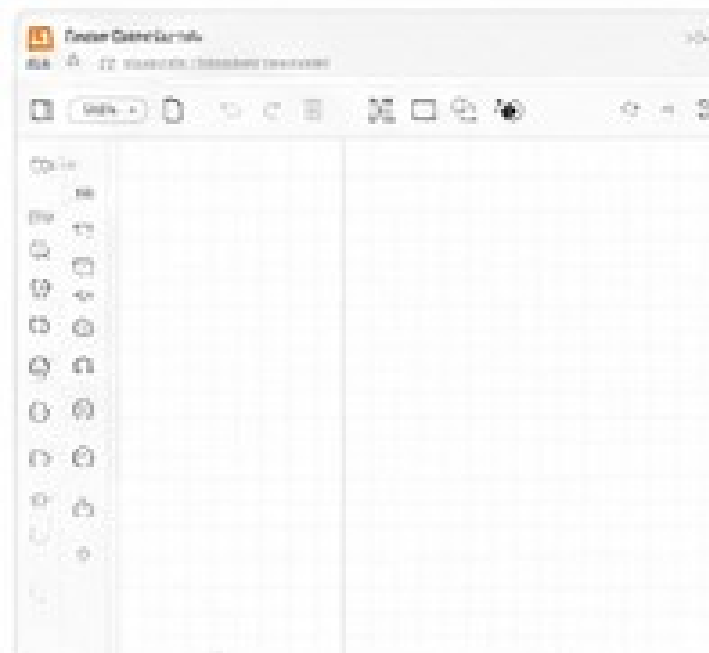
ID	Use Case	Actor	Priority
UC1	Log in	Customer, Manager	High
UC2	View accounts	Customer	High
UC3	Make deposit	Customer	High
UC4	Pay (account / credit card)	Customer	High
UC5	Withdraw	Customer	High
UC6	View all customers	Manager	High
UC7	Add customer	Manager	Medium
UC8	Remove customer	Manager	Medium

Lab 1 Exercises

1 Open Draw.io

Go to <https://app.diagrams.net>

A new blank diagram will open.



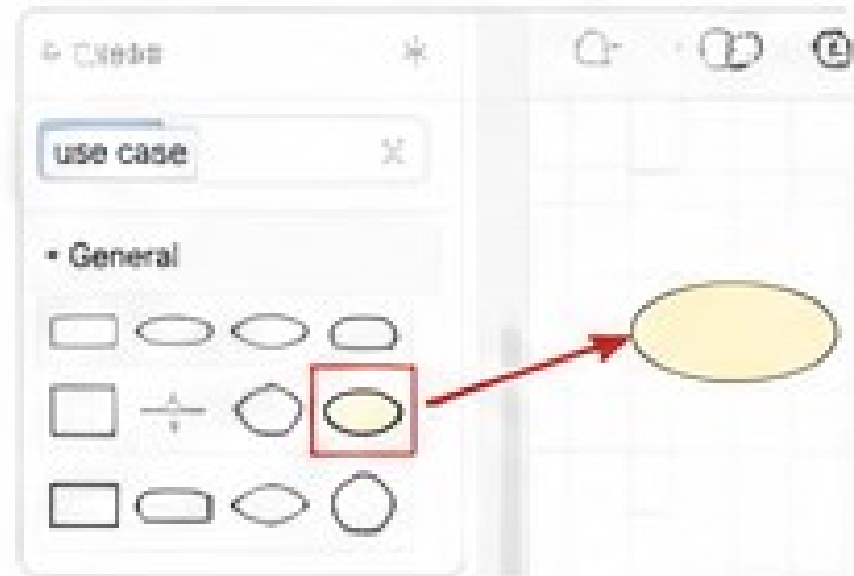
Lab 1 Exercises

2 Add Use Case Shapes

Click Shapes 

Search "use case"

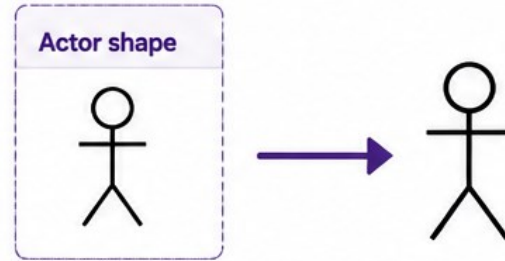
Drag and drop use case ellipses onto the canvas.



Lab 1 Exercises

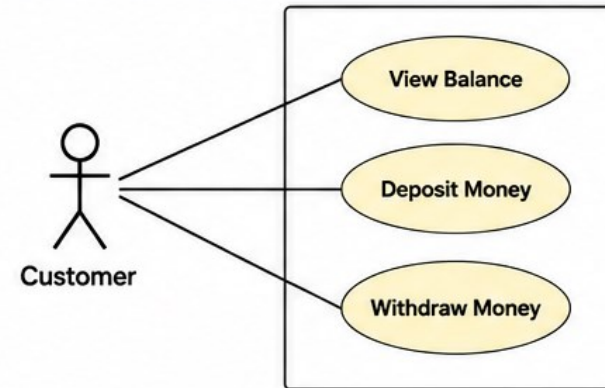
3 Add Actors

Drag the **Actor** shape onto the canvas from the **General** library.



Example

Actor: Customer



Key point

Actors represent people, systems, or external entities that interact with the system.

Lab 1 Exercises

Connect Actors to Use Cases

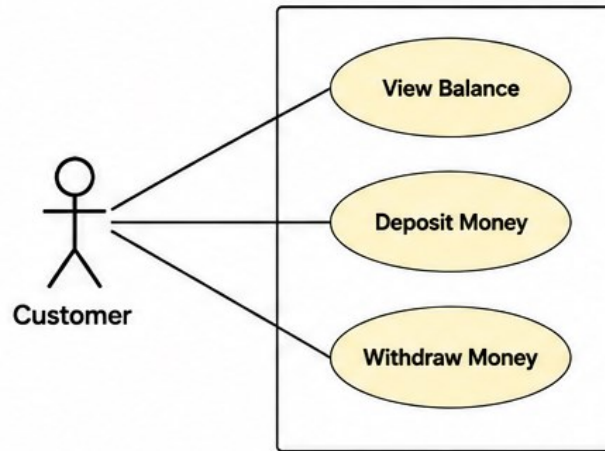
Select the Connector tool



and draw associations (lines) between actors and use cases.

Example

Customer is connected to all use cases that the customer can perform.



Key point

Associations show which actor participates in which use cases.

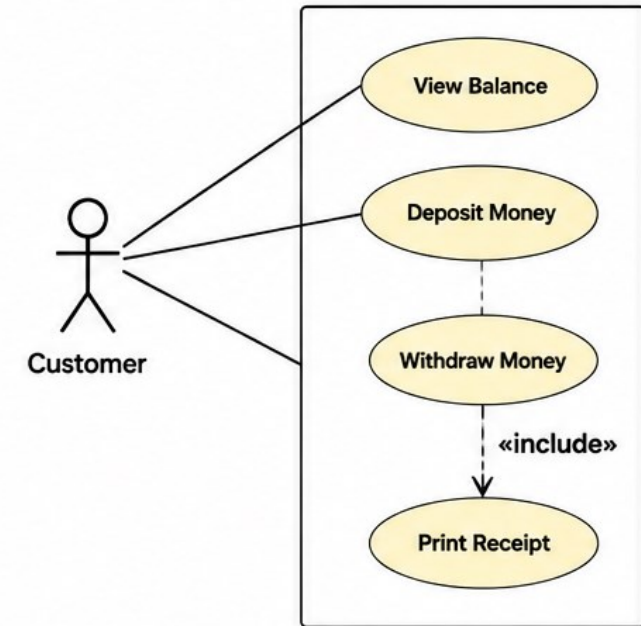
Lab 1 Exercises

6 Add Relationships

Use dashed arrows for «include» or «extend» relationships if required.

Example

The use case “Print Receipt” is included in “Withdraw Money”.

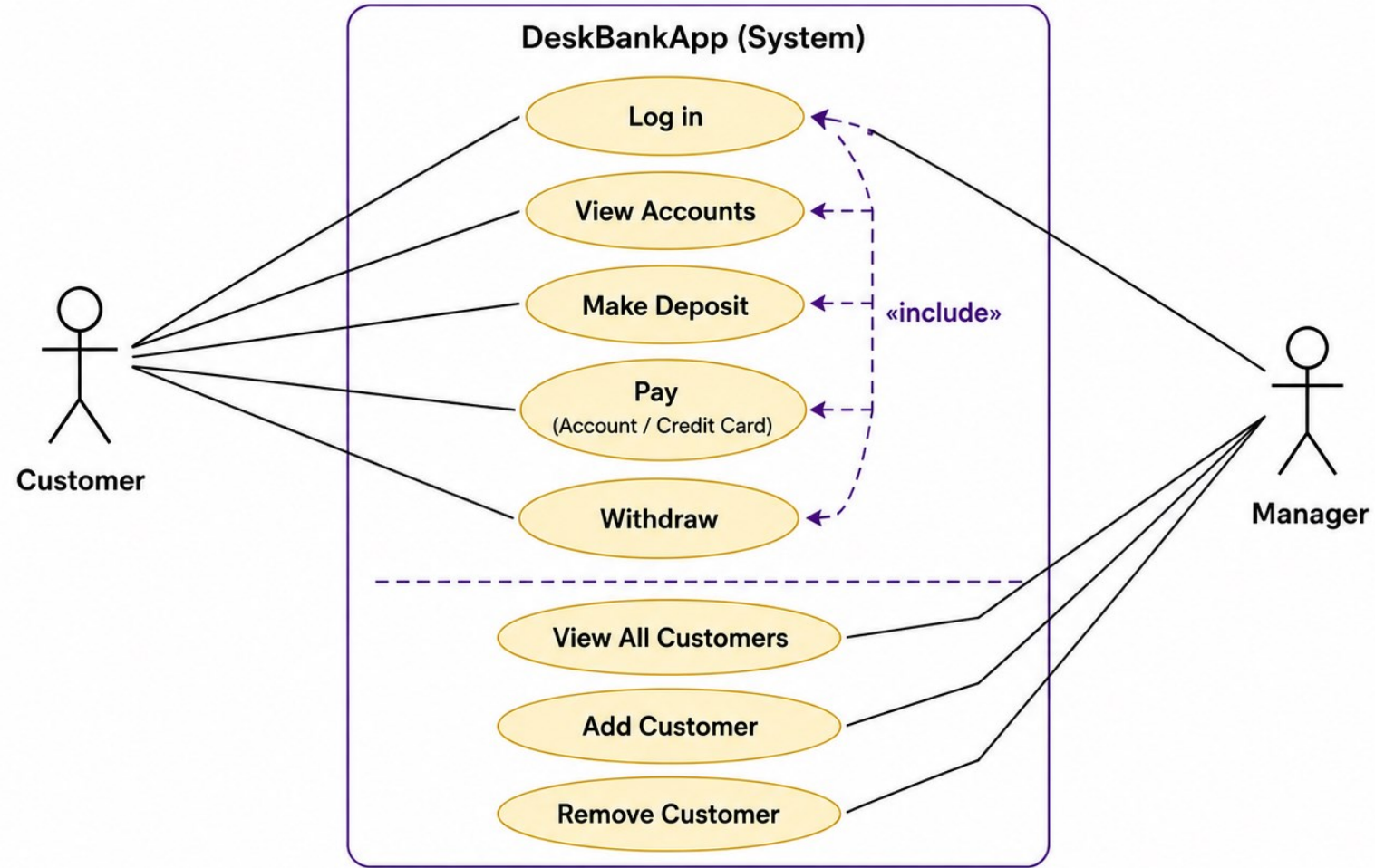


Key point

«include» means the included use case is always performed as part of the base use case.
«extend» adds optional or conditional behavior.

Example Complete Use Case Diagram

Use Case Diagram – DeskBankApp



Legend



Actor

Represents a user or external entity



Use Case

Represents a goal or function provided by the system



Association

Communication between actor and use case



«include»

The use case is always performed as part of another use case



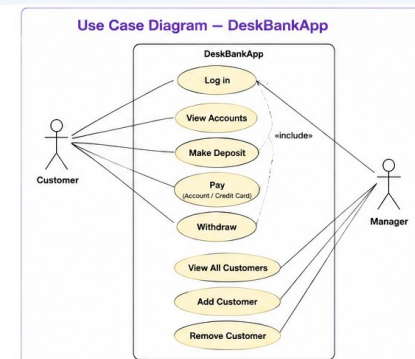
System Boundary

Defines the scope of the system

Lab 1 Exercises

? Questions

1. Who are the external actors in DeskBankApp?
2. Which use cases belong to the Customer?
3. Which use cases are performed by the Manager?
4. What relationship is shown between "Log in" and other use cases?
5. Why do we draw a system boundary in a use case diagram?

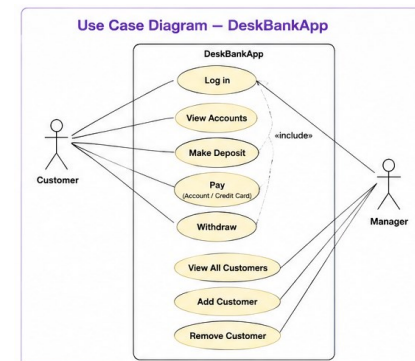


Lab 1 Exercises



Answers (Simple Terms)

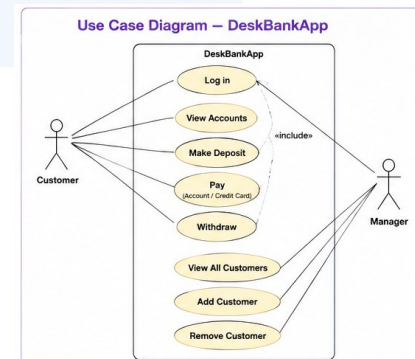
1. Customer and Manager.
2. Log in, View Accounts, Make Deposit, Pay, Withdraw.
3. Log in, View All Customers, Add Customer, Remove Customer.
4. «include» – it means logging in is required to perform those actions.
5. It shows what is inside the system (DeskBankApp) and what is outside.



Lab 1 Exercises

★ Tips

- Use clear names (use verbs).
- Keep the diagram simple.
- Focus on the user goals.
- Validate with stakeholders.
- Update the diagram if requirements change.



Week 1 Summary and Key Takeaways

Every expert programmer was once a beginner. Focus on understanding the concepts, practise regularly, and don't be afraid to ask questions. Small improvements every week lead to big achievements by the end of the semester.

Let's Answer Your Questions



1. **Canvas Inbox** – For academic queries.
2. **Email** – For formal communication and attachments.
3. **Canvas Discussion** – For academic discussions and general queries.
4. **Reminder** – Please follow up if you do not receive a response within 3 business days.



Happy Learning and Bright Futures Ahead!

Dr. Farshid Keivanian

Stay Connected:

[linkedin.com/in/farshid-keivanian](https://www.linkedin.com/in/farshid-keivanian)

github.com/FarshidKeivanian

- *Education is the most powerful weapon you can use to change the world. — Nelson Mandela*